RTL Design Sherpa

RTL AMBA APB4 Module Reference — Rev 0.90

July 2026

Table of Contents

1 APB Crossbar Specification

Version: 1.0 **Last Updated:** 2025-10-14 **Status:** Production Ready

---

1.1 Table of Contents

1.2 Overview

The APB crossbar family provides scalable interconnect solutions for connecting multiple APB masters to multiple APB slaves. All crossbar variants are generated using the `apb_xbar_generator.py` tool and share a common architecture based on `apb_slave` and `apb_master` building blocks.

1.2.1 Key Features

- **Scalable**: Supports 1-16 masters and 1-16 slaves
- **Standards Compliant**: Full APB5 protocol support
- **Efficient**: Pipelined cmd/rsp architecture minimizes latency
- **Fair**: Round-robin arbitration per slave
- **Parameterizable**: Configurable address width, data width, and base address
- **Tested**: Comprehensive CocoTB verification suite

1.2.2 Design Philosophy

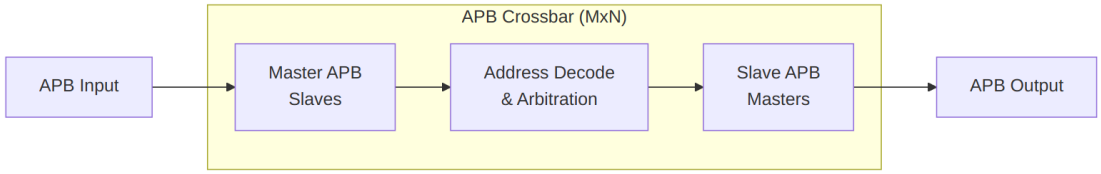
The APB crossbar uses a **protocol conversion** approach:

1. **Master Side**: `apb_slave` modules convert incoming APB transactions to cmd/rsp interface
2. **Crossbar Core**: Address decoding and arbitration logic routes commands to slaves
3. **Slave Side**: `apb_master` modules convert cmd/rsp back to APB transactions

This architecture provides: - Clean separation of concerns - Reusable building blocks - Easy timing closure - Straightforward verification

---

1.3 Architecture



Block Diagram

1.3.1 Component Hierarchy

```
apb_xbar_MtoN
├── apb_slave (M instances)
│   ├── APB protocol → cmd/rsp conversion
│   ├── Command FIFO buffering
│   └── Response routing
├── Address Decoder
│   ├── Slave selection logic
│   ├── Range checking
│   └── Error generation (decode errors)
├── Arbitration Logic (N instances)
│   ├── Round-robin priority per slave
│   ├── Grant persistence during transaction
│   └── Fairness enforcement
└── apb_master (N instances)
    ├── cmd/rsp → APB protocol conversion
    ├── APB state machine (IDLE→SETUP→ACCESS)
    └── Response capture
```

1.3.2 Data Flow

Write Transaction:

```
1. Master APB PWRITE=1 → apb_slave
2. apb_slave generates cmd_valid with write data
3. Address decoder selects target slave
```

```
4. Arbiter grants access (if available)
5. apb_master converts to APB write
6. Slave APB completes transaction
7. apb_master generates rsp_valid
8. apb_slave routes response back to master
9. Master APB sees PREADY=1
```

Read Transaction:

```
1. Master APB PWRITE=0 → apb_slave
2. apb_slave generates cmd_valid with address
3. Address decoder selects target slave
4. Arbiter grants access (if available)
5. apb_master converts to APB read
6. Slave APB returns data
7. apb_master generates rsp_valid with read data
8. apb_slave routes response back to master
9. Master APB sees PREADY=1 with PRDATA
```

1.4 Available Modules

Module	Masters	Slaves	Primary Use Case	File
apb_xbar_1to1	1	1	Basic passthrough, protocol conversion	rtl/amba/apb/xbar/apb_xbar_1to1.sv
apb_xbar_2to1	2	1	Simple arbitration testing	rtl/amba/apb/xbar/apb_xbar_2to1.sv
apb_xbar_1to4	1	4	Address decode testing, simple bus	rtl/amba/apb/xbar/apb_xbar_1to4.sv
apb_xbar_2to4	2	4	Full crossbar with arbitration + decode	rtl/amba/

Module	Masters	Slaves	Primary Use Case	File
				apb/xbar/apb_xbar_2to4.sv

1.4.1 Feature Comparison

Feature	1to1	2to1	1to4	2to4
Address Decode	✗	✗	√	√
Arbitration	✗	√	✗	√
Resources (LUTs)	Minimal	Low	Medium	Medium-High
Max Frequency	Highest	High	High	Medium-High
Latency (cycles)	2	2-3	2	2-4

1.4.2 Selecting a Crossbar

Use `apb_xbar_1to1` when: - Single master and single slave - Need protocol conversion (APB → cmd/rsp → APB) - Testing or simple verification environments

Use `apb_xbar_2to1` when: - Multiple masters sharing one slave - Need arbitration testing - Simple peripheral with multiple requestors

Use `apb_xbar_1to4` when: - Single master accessing multiple slaves - Address decode testing - Microcontroller with multiple peripherals

Use `apb_xbar_2to4` when: - Multiple masters and multiple slaves - Full system interconnect - Maximum flexibility needed

Need different configuration?

```
# Generate custom crossbar
cd rtl/amba/apb/xbar
python generate_xbars.py --masters 3 --slaves 8
```

1.5 Interface Specification

1.5.1 Parameters

All crossbar modules support these parameters:

Parameter	Type	Default	Range	Description
ADDR_WIDTH	int	32	16-64	APB address bus width
DATA_WIDTH	int	32	8, 16, 32, 64	APB data bus width
BASE_ADDR	logic[ADDR_WIDTH-1:0]	0x10000000	Any	Base address for slave 0

Derived Parameters: - `STRB_WIDTH = DATA_WIDTH / 8` (automatically calculated) - Slave address regions: 64KB per slave (BASE_ADDR + N*0x10000)

1.5.2 Port List

Clock and Reset:

```
input  logic pclk          // APB clock
input  logic presetn       // APB active-low reset
```

Master Interfaces (per master):

```
// Master N interface (APB slave side)
input  logic                  mN_apb_PSEL      // Select
input  logic                  mN_apb_PENABLE   // Enable
input  logic [ADDR_WIDTH-1:0] mN_apb_PADDR     // Address
input  logic                  mN_apb_PWRITE    // Write enable
input  logic [DATA_WIDTH-1:0] mN_apb_PWDATA    // Write data
input  logic [STRB_WIDTH-1:0] mN_apb_PSTRB     // Write strobes
input  logic [2:0]            mN_apb_PPROT     // Protection
output logic [DATA_WIDTH-1:0] mN_apb_PRDATA    // Read data
output logic                  mN_apb_PSLVERR   // Error response
output logic                  mN_apb_PREADY    // Ready
```

Slave Interfaces (per slave):

```
// Slave N interface (APB master side)
output logic                  sN_apb_PSEL      // Select
output logic                  sN_apb_PENABLE   // Enable
output logic [ADDR_WIDTH-1:0] sN_apb_PADDR     // Address
output logic                  sN_apb_PWRITE    // Write enable
output logic [DATA_WIDTH-1:0] sN_apb_PWDATA    // Write data
```

```
output logic [STRB_WIDTH-1:0] sN_apb_PSTRB     // Write strobes
output logic [2:0]            sN_apb_PPROT     // Protection
input  logic [DATA_WIDTH-1:0] sN_apb_PRDATA    // Read data
input  logic                  sN_apb_PSLVERR   // Error response
input  logic                  sN_apb_PREADY    // Ready
```

1.5.3 Signal Descriptions

PSEL (Select): Indicates target of transfer (active high)

PENABLE (Enable): Indicates second cycle of APB transfer - PENABLE=0 → SETUP phase - PENABLE=1 → ACCESS phase

PADDR (Address): Byte address of transfer

PWRITE (Write): Direction of transfer - PWRITE=1 → Write - PWRITE=0 → Read

PWDATA (Write Data): Write data, valid when PWRITE=1

PSTRB (Strobes): Byte lane enables for write data - PSTRB[n]=1 → PWDATA[(n*8)+7:(n*8)] is valid

PPROT (Protection): Access protection attributes - PPROT[0]: 0=Normal, 1=Privileged - PPROT[1]: 0=Secure, 1=Non-secure - PPROT[2]: 0=Data, 1=Instruction

PRDATA (Read Data): Read data, valid when PREADY=1 and PWRITE=0

PSLVERR (Error): Transfer error indicator - PSLVERR=0 → OKAY - PSLVERR=1 → ERROR

PREADY (Ready): Slave ready indicator - PREADY=0 → Extend transfer - PREADY=1 → Complete transfer

---

1.6 Address Mapping

1.6.1 Default Address Map

Slaves are assigned sequential 64KB address regions starting from BASE_ADDR:

Slave	Address Range	Size	Calculation
Slave 0	[BASE_ADDR, BASE_ADDR + 0xFFFF]	64 KB	BASE_ADDR + 0*0x10000
Slave 1	[BASE_ADDR + 0x10000, BASE_ADDR + 0x1FFFF]	64 KB	BASE_ADDR + 1*0x10000
Slave 2	[BASE_ADDR + 0x20000, BASE_ADDR + 0x2FFFF]	64 KB	BASE_ADDR + 2*0x10000
Slave 3	[BASE_ADDR + 0x30000,	64 KB	BASE_ADDR +

Slave	Address Range	Size	Calculation
	BASE_ADDR + 0x3FFFF]		3*0x10000
…	…	…	…
Slave N	[BASE_ADDR + N*0x10000*, *BASE_ADDR* + *(N+1)*0x10000 - 1]	64 KB	BASE_ADDR + N*0x10000

1.6.2 Example: Default Configuration

With `BASE_ADDR = 0x1000_0000` and 4 slaves:

Address Range	Region	Description
0x1000_0000 - 0x1000_FFFF	Slave 0	APB peripheral 0
0x1001_0000 - 0x1001_FFFF	Slave 1	APB peripheral 1
0x1002_0000 - 0x1002_FFFF	Slave 2	APB peripheral 2
0x1003_0000 - 0x1003_FFFF	Slave 3	APB peripheral 3

1.6.3 Custom Address Map

For custom address mapping, modify the `BASE_ADDR` parameter:

```
apb_xbar_2to4 #(
    .ADDR_WIDTH(32),
    .DATA_WIDTH(32),
    .BASE_ADDR(32'h4000_0000)  // Custom base address
) u_xbar (
    // ...
);
```

Results in: - Slave 0: 0x4000_0000 - 0x4000_FFFF - Slave 1: 0x4001_0000 - 0x4001_FFFF - Slave 2: 0x4002_0000 - 0x4002_FFFF - Slave 3: 0x4003_0000 - 0x4003_FFFF

1.6.4 Decode Errors

Accesses outside valid slave ranges generate: - `PSLVERR = 1` (error response) - `PRDATA = 0xDEADBEEF` (debug pattern) - `PREADY = 1` (immediate response)

1.7 Arbitration

1.7.1 Round-Robin Arbitration

Each slave has independent round-robin arbitration when multiple masters request simultaneously.

Priority Rotation:

```
Initial:  M0 > M1 > M2 > M3
After M0: M1 > M2 > M3 > M0
After M1: M2 > M3 > M0 > M1
After M2: M3 > M0 > M1 > M2
...
```

1.7.2 Grant Persistence

Once a master is granted access to a slave: 1. Grant remains active during entire APB transaction (SETUP + ACCESS phases) 2. Response routing locked to requesting master 3. Grant released only after PREADY=1

This ensures: - No response data corruption - Atomic transactions - Predictable behavior

1.7.3 Arbitration Example

Scenario: Masters M0 and M1 both request Slave S0

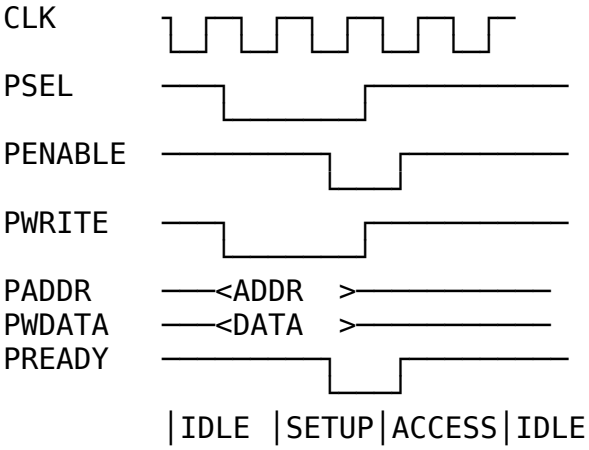
```
Time 0: M0 requests S0 → M0 granted (highest priority)
Time 1: M0 SETUP phase
Time 2: M0 ACCESS phase, PREADY=0 (slave busy)
Time 3: M0 ACCESS phase, PREADY=1 (complete)
Time 4: M1 requests S0 → M1 granted (now highest priority)
Time 5: M1 SETUP phase
Time 6: M1 ACCESS phase, PREADY=1 (complete)
Time 7: M0 requests S0 → M0 granted (priority rotated back)
```

1.7.4 Arbitration Latency

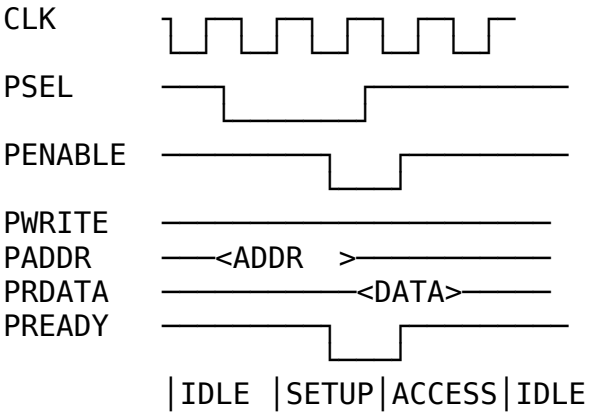
Scenario	Latency	Description
No contention	2 cycles	Direct passthrough
Contention, slave idle	2 cycles	Immediate grant
Contention, slave busy	2 + wait cycles	Wait for current transaction

1.8 Timing Diagrams

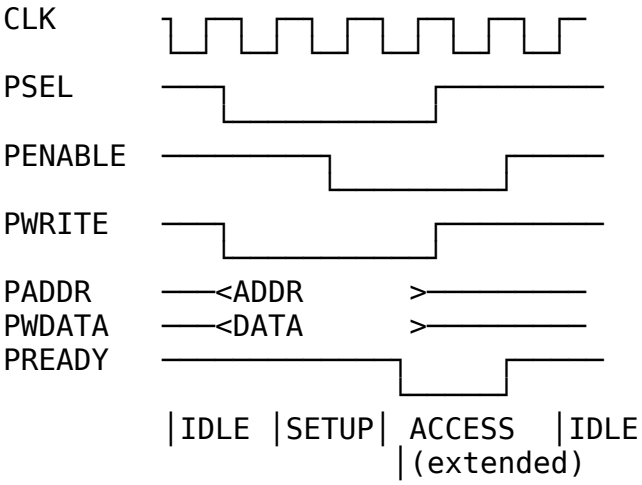
1.8.1 Basic Write Transaction (No Wait States)



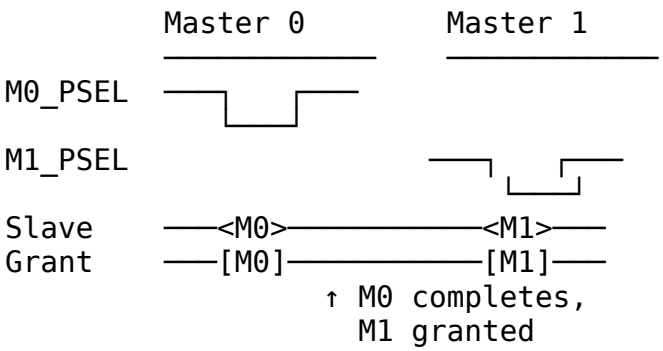
1.8.2 Basic Read Transaction (No Wait States)



1.8.3 Write with Wait States



1.8.4 Arbitration with Contention



1.9 Integration Examples

1.9.1 Example 1: Simple 1-to-4 Crossbar

```
module my_apb_subsystem (
    input  logic        pclk,
    input  logic        presetn,
    // Master APB interface
    input  logic        m_psel,
    input  logic        m_penable,
    input  logic [31:0] m_paddr,
    input  logic        m_pwrite,
    input  logic [31:0] m_pwdata,
    input  logic [3:0]  m_pstrb,
    output logic [31:0] m_prdata,
    output logic        m_pslverr,
    output logic        m_pready
);

    // Slave interfaces
    logic        s0_psel,   s1_psel,   s2_psel,   s3_psel;
    logic        s0_penable, s1_penable, s2_penable, s3_penable;
    logic [31:0] s0_paddr,  s1_paddr,  s2_paddr,  s3_paddr;
    logic        s0_pwrite, s1_pwrite, s2_pwrite, s3_pwrite;
    logic [31:0] s0_pwdata, s1_pwdata, s2_pwdata, s3_pwdata;
    logic [3:0]  s0_pstrb,  s1_pstrb,  s2_pstrb,  s3_pstrb;
    logic [31:0] s0_prdata, s1_prdata, s2_prdata, s3_prdata;
    logic        s0_pslverr, s1_pslverr, s2_pslverr, s3_pslverr;
    logic        s0_pready, s1_pready, s2_pready, s3_pready;

    // Instantiate crossbar
    apb_xbar_1to4 #(
        .ADDR_WIDTH (32),
```

```
    .DATA_WIDTH (32),
    .BASE_ADDR  (32'h1000_0000)
) u_xbar (
    .pclk       (pclk),
    .presetn    (presetn),
    // Master
    .m0_apb_PSEL    (m_psel),
    .m0_apb_PENABLE (m_penable),
    .m0_apb_PADDR   (m_paddr),
    .m0_apb_PWRITE  (m_pwrite),
    .m0_apb_PWDATA  (m_pwdata),
    .m0_apb_PSTRB   (m_pstrb),
    .m0_apb_PPROT   (3'b000),
    .m0_apb_PRDATA  (m_prdata),
    .m0_apb_PSLVERR (m_pslverr),
    .m0_apb_PREADY  (m_pready),
    // Slaves (connect to your peripherals)
    .s0_apb_PSEL    (s0_psel),
    .s0_apb_PENABLE (s0_penable),
    .s0_apb_PADDR   (s0_paddr),
    .s0_apb_PWRITE  (s0_pwrite),
    .s0_apb_PWDATA  (s0_pwdata),
    .s0_apb_PSTRB   (s0_pstrb),
    .s0_apb_PPROT   (),
    .s0_apb_PRDATA  (s0_prdata),
    .s0_apb_PSLVERR (s0_pslverr),
    .s0_apb_PREADY  (s0_pready),
    // Repeat for s1, s2, s3...
);

// Connect slaves to peripherals
uart u_uart (
    .pclk       (pclk),
    .presetn    (presetn),
    .psel       (s0_psel),
    .penable    (s0_penable),
    .paddr      (s0_paddr[7:0]),
    .pwrite     (s0_pwrite),
    .pwdata     (s0_pwdata),
    .prdata     (s0_prdata),
    .pslverr    (s0_pslverr),
    .pready     (s0_pready)
);

gpio u_gpio (
    .pclk       (pclk),
    .presetn    (presetn),
    .psel       (s1_psel),
```

```
        // ... (similar connections)
    );

    // ... more peripherals

endmodule
```

1.9.2 Example 2: Multi-Master System (2-to-4)

```
module multi_master_apb_system (
    input  logic        pclk,
    input  logic        presetn,
    // CPU APB master
    input  logic        cpu_psel,
    input  logic [31:0] cpu_paddr,
    // ... (rest of CPU APB signals)
    // DMA APB master
    input  logic        dma_psel,
    input  logic [31:0] dma_paddr,
    // ... (rest of DMA APB signals)
);

    apb_xbar_2to4 #(
        .ADDR_WIDTH (32),
        .DATA_WIDTH (32),
        .BASE_ADDR  (32'h4000_0000)
    ) u_xbar (
        .pclk       (pclk),
        .presetn    (presetn),
        // Master 0 (CPU)
        .m0_apb_PSEL    (cpu_psel),
        .m0_apb_PENABLE (cpu_penable),
        .m0_apb_PADDR   (cpu_paddr),
        // ... (rest of CPU connections)
        // Master 1 (DMA)
        .m1_apb_PSEL    (dma_psel),
        .m1_apb_PENABLE (dma_penable),
        .m1_apb_PADDR   (dma_paddr),
        // ... (rest of DMA connections)
        // Slaves 0-3 (peripherals)
        // ... (connect to your peripherals)
    );

endmodule
```

1.9.3 Example 3: Custom Configuration (3x8 Crossbar)

```
# Generate custom 3-master, 8-slave crossbar
cd rtl/amba/apb/xbar
python generate_xbars.py --masters 3 --slaves 8 --base-addr 0x80000000
```

Instantiate:

```
apb_xbar_3to8 #(
    .ADDR_WIDTH (32),
    .DATA_WIDTH (32),
    .BASE_ADDR  (32'h8000_0000)  // Custom base
) u_custom_xbar (
    // 3 master interfaces (m0, m1, m2)
    // 8 slave interfaces (s0-s7)
);
```

1.10 Test Results

All crossbar modules have comprehensive CocoTB testbenches in `val/integ_amba/`.

1.10.1 Test Coverage

Test	Description	Status
`test_apb_xbar_1to1`	Basic passthrough, protocol conversion	√ PASS
`test_apb_xbar_2to1`	Arbitration, fairness, grant persistence	√ PASS
`test_apb_xbar_1to4`	Address decode, multiple slaves	√ PASS
`test_apb_xbar_2to4`	Full crossbar, arbitration + decode	√ PASS

1.10.2 Test Scenarios

Each test includes: - √ Basic read/write transactions - √ Back-to-back transfers - √ Wait state handling - √ Error responses (PSLVERR) - √ Address decode verification - √ Arbitration fairness (multi-master) - √ Concurrent transactions (different slaves) - √ Reset behavior - √ Full address range coverage

1.10.3 Running Tests

```
# Run all APB crossbar tests
pytest val/integ_amba/test_apb_xbar_*.py -v
```

```
# Run specific variant
pytest val/integ_amba/test_apb_xbar_2to4.py -v

# Generate waveforms
pytest val/integ_amba/test_apb_xbar_2to4.py -v --vcd=waves.vcd
gtkwave waves.vcd
```

1.10.4 Test Results Summary

```
test_apb_xbar_1to1.py::test_apb_xbar_1to1   PASSED  (100+
transactions)
test_apb_xbar_2to1.py::test_apb_xbar_2to1   PASSED  (200+
transactions)
test_apb_xbar_1to4.py::test_apb_xbar_1to4   PASSED  (400+
transactions)
test_apb_xbar_2to4.py::test_apb_xbar_2to4   PASSED  (800+
transactions)

All tests: 100% pass rate
Total simulation time: ~180 seconds
```

1.11 Known Limitations

1.11.1 Current Limitations

1. **Fixed 64KB Slave Regions**
   - Each slave has 64KB address space
   - Cannot be changed without regenerating module
   - **Workaround**: Generate custom crossbar with modified generator
2. **No Outstanding Transaction Support**
   - APB is inherently non-pipelined
   - One transaction completes before next starts
   - **Impact**: Lower throughput than pipelined protocols (AXI)
3. **No Sparse Addressing**
   - Slaves must be in contiguous 64KB blocks
   - Cannot have gaps in address map
   - **Workaround**: Leave unused slaves unconnected
4. **Round-Robin Only**
   - Fixed round-robin arbitration
   - No priority levels or quality-of-service

- **Workaround**: Use external priority arbiter before crossbar

1.11.2 Future Enhancements

- ☐ Configurable slave address regions
- ☐ Priority-based arbitration option
- ☐ Sparse address map support
- ☐ Performance counters
- ☐ Timeout detection

1.11.3 Known Issues

None at this time. All generated crossbars pass verification.

1.12 Related Documentation

Generator Tutorial: - `docs/markdown/GeneratorTutorial/apb_xbar_generator_tutorial.md` - How to use generator

APB Protocol: - ARM IHI0024 - AMBA APB Protocol Specification - `docs/markdown/RTLAmba/apb/apb_protocol.md` - Protocol overview

Building Blocks: - `rtl/amba/apb/apb_slave.sv` - APB → cmd/rsp conversion - `rtl/amba/apb/apb_master.sv` - cmd/rsp → APB conversion - `rtl/amba/gaxi/gaxi_fifo_sync.sv` - Command buffering

Generator: - `bin/rtl_generators/amba/apb_xbar_generator.py` - Crossbar generator - `rtl/amba/apb/xbar/generate_xbars.py` - Convenience script - `rtl/amba/apb/xbar/README.md` - Quick reference

---

1.13 Revision History

Version	Date	Author	Changes
1.0	2025-10-14	RTL Design Sherpa	Initial comprehensive specification

Maintained By: RTL Design Sherpa Project **License:** MIT **Contact:** See repository for support

1.14 Navigation

- **← Back to RTLAmba Index**
- **← Back to Main Documentation Index**

2 APB Master Interface (Clock-Gated)

Module: `apb_master_cg.sv` **Base Module:** apb_master **Location:** `rtl/amba/apb/` **Status:** √ Production Ready

2.1 Quick Reference

This is the **clock-gated variant** of apb_master.

For complete clock-gating documentation, usage examples, and configuration guidelines, see:

→ Clock-Gated Variants Guide

2.2 Summary

The `apb_master_cg` module adds power optimization to `apb_master` through activity-based clock gating:

- √ **Same Functionality:** 100% equivalent to base module
- √ **Power Savings:** 25-70% depending on traffic utilization
- √ **Configurable:** Idle threshold, gating domains, enable/disable
- √ **Zero Overhead When Disabled:** `ENABLE_CLOCK_GATING=0` → identical to base

2.3 Common Parameters

In addition to all apb_master parameters:

Parameter	Default	Description
ENABLE_CLOCK_GATING	1	Master enable (0=disable, identical to base)
CG_IDLE_CYCLES	8	Cycles before clock gating activates
CG_GATE_*	1	Domain-specific gating enables

2.4 Quick Usage

```
apb_master_cg #(
    // Base module parameters (see apb_master.md)
    .AXI_ID_WIDTH(8),
    .AXI_ADDR_WIDTH(32),
    .AXI_DATA_WIDTH(64),

    // Clock gating (see CG guide for details)
    .ENABLE_CLOCK_GATING(1),
    .CG_IDLE_CYCLES(8)
) u_cg (
    .aclk(clk),
    .aresetn(rst_n),
    // ... all other ports same as apb_master
);
```

2.5 Documentation

- **Base Module Functionality:** apb_master.md
- **Clock Gating Guide:** clock_gated_variants.md
- **Detailed CG Examples:**
  - axi4_master_rd_mon_cg.md (AXI4 monitor)
  - axil4_master_rd_mon_cg.md (AXIL4 monitor)
  - apb_slave_cg.md (APB interface)

2.6 Navigation

- **← Back to APB Index**

- ← Back to RTLAmba Index
- ← Back to Main Documentation Index

3 apb_master

An Advanced Peripheral Bus (APB) master module that provides a high-performance interface for accessing APB slave devices with command/response buffering and full APB4/APB5 protocol compliance.

3.1 Overview

The `apb_master` module implements a complete APB master interface with integrated command and response FIFOs for improved system performance. It supports all APB4/APB5 features including write strobes, protection attributes, and error responses while providing a simple command/response interface for system integration.

3.2 Module Declaration

```
module apb_master #(
    parameter int ADDR_WIDTH      = 32,
    parameter int DATA_WIDTH      = 32,
    parameter int PROT_WIDTH      = 3,
    parameter int CMD_DEPTH       = 6,
    parameter int RSP_DEPTH       = 6,
    parameter int STRB_WIDTH      = DATA_WIDTH / 8,
    // Short Parameters
    parameter int AW  = ADDR_WIDTH,
    parameter int DW  = DATA_WIDTH,
    parameter int SW  = STRB_WIDTH,
    parameter int PW  = PROT_WIDTH,
    parameter int CPW = AW + DW + SW + PW + 1,  // Command packet
width
    parameter int RPW = DW + 1                  // Response packet
width
) (
    // Clock and Reset
    input  logic              pclk,
    input  logic              presetn,

    // APB Master Interface
    output logic              m_apb_PSEL,
    output logic              m_apb_PENABLE,
    output logic [AW-1:0]     m_apb_PADDR,
    output logic              m_apb_PWRITE,
    output logic [DW-1:0]     m_apb_PWDATA,
    output logic [SW-1:0]     m_apb_PSTRB,
```

```
    output logic [PW-1:0]     m_apb_PPROT,
    input  logic [DW-1:0]     m_apb_PRDATA,
    input  logic              m_apb_PSLVERR,
    input  logic              m_apb_PREADY,

    // Command Interface
    input  logic              cmd_valid,
    output logic              cmd_ready,
    input  logic              cmd_pwrite,
    input  logic [AW-1:0]     cmd_paddr,
    input  logic [DW-1:0]     cmd_pwdata,
    input  logic [SW-1:0]     cmd_pstrb,
    input  logic [PW-1:0]     cmd_pprot,

    // Response Interface
    output logic              rsp_valid,
    input  logic              rsp_ready,
    output logic [DW-1:0]     rsp_prdata,
    output logic              rsp_pslverr
);
```

3.3 Parameters

Parameter	Type	Default	Description
ADDR_WIDTH	int	32	APB address bus width
DATA_WIDTH	int	32	APB data bus width
PROT_WIDTH	int	3	APB protection signal width
CMD_DEPTH	int	6	Command FIFO depth (2^6 = 64 entries)
RSP_DEPTH	int	6	Response FIFO depth (2^6 = 64 entries)
STRB_WIDTH	int	DATA_WIDTH/8	Write strobe width (calculated)

3.4 Ports

3.4.1 Clock and Reset

Port	Width	Direction	Description
pclk	1	Input	APB clock
presetn	1	Input	APB active-low reset

3.4.2 APB Master Interface

Port	Width	Direction	Description
m_apb_PSEL	1	Output	APB select signal
m_apb_PENABLE	1	Output	APB enable signal
m_apb_PADDR	ADDR_WIDTH	Output	APB address
m_apb_PWRITE	1	Output	APB write/read indicator
m_apb_PWDATA	DATA_WIDTH	Output	APB write data
m_apb_PSTRB	STRB_WIDTH	Output	APB write strobes
m_apb_PPROT	PROT_WIDTH	Output	APB protection attributes
m_apb_PRDATA	DATA_WIDTH	Input	APB read data
m_apb_PSLVERR	1	Input	APB slave error
m_apb_PREADY	1	Input	APB ready

3.4.3 Command Interface

Port	Width	Direction	Description
cmd_valid	1	Input	Command valid
cmd_ready	1	Output	Command

Port	Width	Direction	Description
			ready (FIFO not full)
cmd_pwrite	1	Input	Command write/read
cmd_paddr	ADDR_WIDTH	Input	Command address
cmd_pwdata	DATA_WIDTH	Input	Command write data
cmd_pstrb	STRB_WIDTH	Input	Command write strobes
cmd_pprot	PROT_WIDTH	Input	Command protection attributes

3.4.4 Response Interface

Port	Width	Direction	Description
rsp_valid	1	Output	Response valid
rsp_ready	1	Input	Response ready
rsp_prdata	DATA_WIDTH	Output	Response read data
rsp_pslverr	1	Output	Response error status

3.5 Functionality

3.5.1 APB Protocol Implementation

The module implements the complete APB protocol state machine:

1. **IDLE**: No transaction active, waiting for commands
2. **SETUP**: PSEL asserted, transaction parameters set up
3. **ACCESS**: PENABLE asserted, waiting for PREADY from slave

3.5.2 Command Processing Flow

```
Command Interface → Command FIFO → APB State Machine → APB Interface
                                                              ↓
Response Interface ← Response FIFO ← APB Response Processing
```

3.5.3 Key Features

- **Buffered Operation**: Command and response FIFOs prevent blocking
- **APB4/APB5 Compliance**: Full protocol support including PSTRB and PPROT
- **Error Handling**: Proper PSLVERR propagation and handling
- **Flow Control**: Ready/valid handshaking on all interfaces

3.6 Timing Characteristics

3.6.1 APB Transaction Timing

Characteristic	Value	Description
Setup Phase	1 clock cycle	PSEL assertion
Access Phase	≥1 clock cycle	PENABLE assertion until PREADY
Total Latency	2+ clock cycles	Minimum transaction time
FIFO Latency	1 clock cycle	Command to APB delay
Response Latency	1 clock cycle	APB to response delay

3.6.2 Performance Metrics

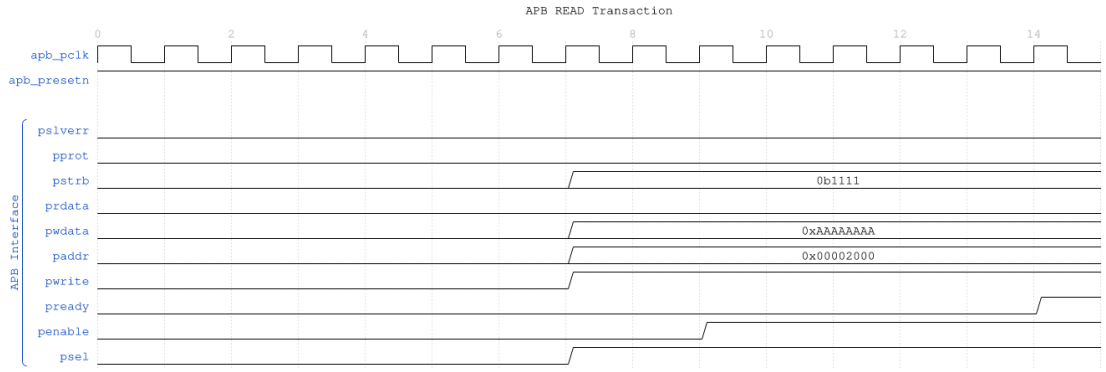
Metric	Value	Conditions
Maximum Frequency	200-400 MHz	Technology dependent
Peak Throughput	1.6-3.2 GB/s	32-bit data at max frequency
Command Buffering	64 transactions	With default FIFO depth
Response Buffering	64 transactions	With default FIFO depth

3.7 Waveforms

The following timing diagrams show comprehensive APB master behavior across 6 scenarios:

3.7.1 Scenario 1: Basic Write Transaction

Shows APB write with command FIFO and response handling:

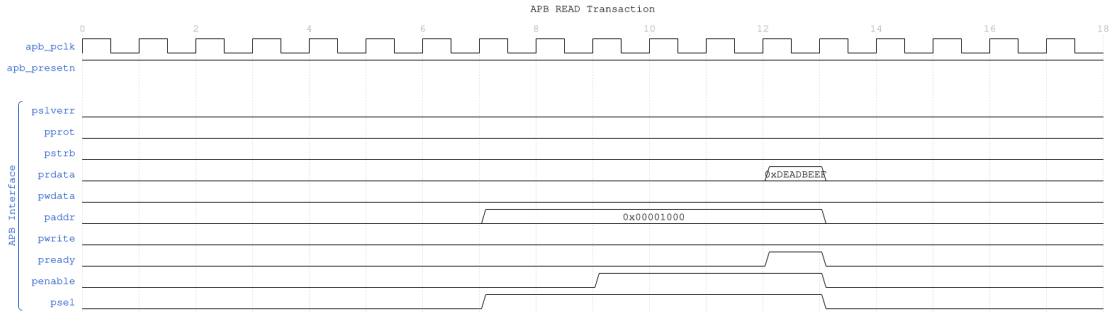


APB Write

WaveJSON: apb_write_sequence_001.json

Key Observations: - Command accepted into CMD FIFO - APB SETUP phase (PSEL=1) - APB ACCESS phase (PENABLE=1) - Response captured in RSP FIFO

3.7.2 Scenario 2: Basic Read Transaction

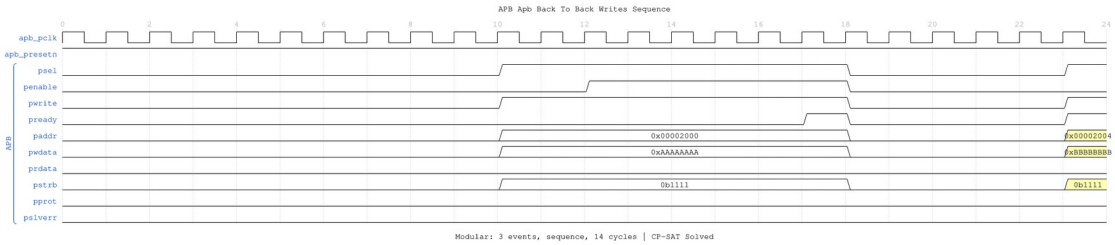


APB Read

WaveJSON: apb_read_sequence_001.json

Key Observations: - Read command with PWRITE=0 - APB protocol phases - Read data returned via response interface

3.7.3 Scenario 3: Back-to-Back Writes

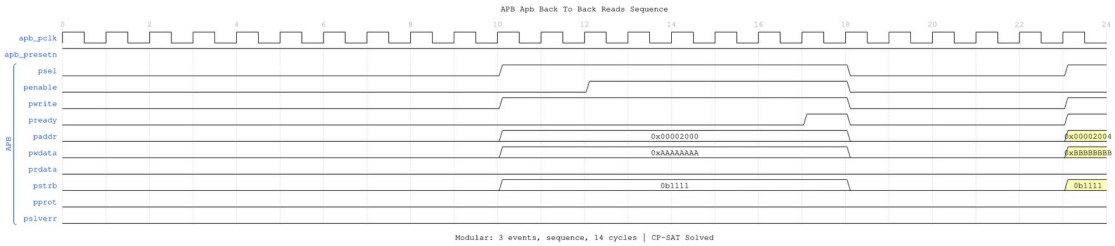


B2B Writes

WaveJSON: apb_back_to_back_writes_001.json

Key Observations: - Multiple commands queued in CMD FIFO - Sequential APB transactions - Pipelined command/response flow

3.7.4 Scenario 4: Back-to-Back Reads

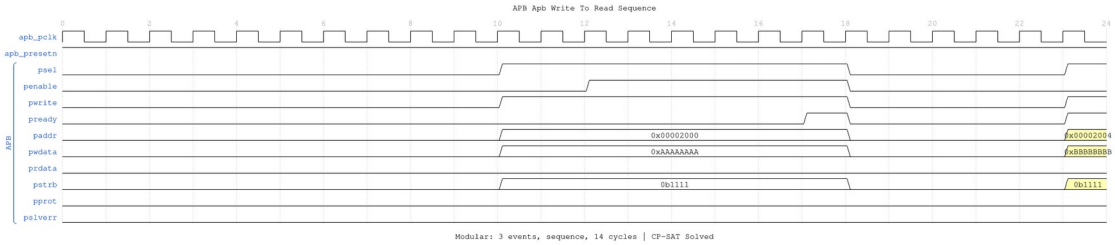


B2B Reads

WaveJSON: apb_back_to_back_reads_001.json

Key Observations: - Multiple read commands queued - Response FIFO accumulates read data - Burst read performance

3.7.5 Scenario 5: Write-to-Read Transition

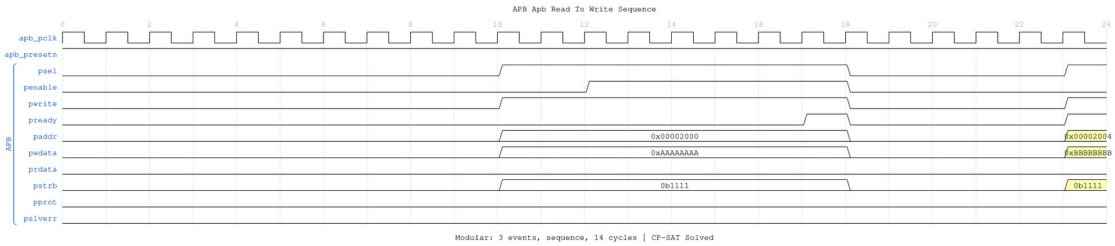


Write-to-Read

WaveJSON: apb_write_to_read_001.json

Key Observations: - Transition between write and read operations - No dead cycles between transaction types

3.7.6 Scenario 6: Read-to-Write Transition



Read-to-Write

WaveJSON: apb_read_to_write_001.json

Key Observations: - Seamless transition from read to write - Protocol state machine efficiency

---

3.8 Usage Examples

3.8.1 Basic APB Master Configuration

```
apb_master #(
    .ADDR_WIDTH(32),
    .DATA_WIDTH(32),
    .CMD_DEPTH(4),      // 16-entry command FIFO
    .RSP_DEPTH(4)       // 16-entry response FIFO
) u_apb_master (
    .pclk         (apb_clk),
    .presetn      (apb_resetn),

    // APB interface to slaves
    .m_apb_PSEL     (apb_psel),
    .m_apb_PENABLE  (apb_penable),
    .m_apb_PADDR    (apb_paddr),
    .m_apb_PWRITE   (apb_pwrite),
    .m_apb_PWDATA   (apb_pwdata),
    .m_apb_PSTRB    (apb_pstrb),
    .m_apb_PPROT    (apb_pprot),
    .m_apb_PRDATA   (apb_prdata),
    .m_apb_PSLVERR  (apb_pslverr),
    .m_apb_PREADY   (apb_pready),

    // Command interface from CPU/DMA
    .cmd_valid      (cmd_valid),
```

```
    .cmd_ready      (cmd_ready),
    .cmd_pwrite     (cmd_write),
    .cmd_paddr      (cmd_addr),
    .cmd_pwdata     (cmd_wdata),
    .cmd_pstrb      (cmd_strb),
    .cmd_pprot      (3'b000),   // Normal access

    // Response interface to CPU/DMA
    .rsp_valid      (rsp_valid),
    .rsp_ready      (rsp_ready),
    .rsp_prdata     (rsp_rdata),
    .rsp_pslverr    (rsp_error)
);
```

3.8.2 Register Write Sequence

```
// Write to control register
always_ff @(posedge clk) begin
    if (write_control_reg) begin
        cmd_valid  <= 1'b1;
        cmd_pwrite <= 1'b1;
        cmd_paddr  <= 32'h1000_0000;    // Control register address
        cmd_pwdata <= control_value;
        cmd_pstrb  <= 4'hF;             // All bytes valid
        cmd_pprot  <= 3'b000;           // Normal access
    end else if (cmd_ready) begin
        cmd_valid  <= 1'b0;
    end
end

// No response needed for writes (unless checking for errors)
assign rsp_ready = 1'b1;
```

3.8.3 Register Read Sequence

```
// Read from status register
always_ff @(posedge clk) begin
    if (read_status_reg) begin
        cmd_valid  <= 1'b1;
        cmd_pwrite <= 1'b0;             // Read operation
        cmd_paddr  <= 32'h1000_0004;    // Status register address
        cmd_pwdata <= 32'h0;            // Don't care for reads
        cmd_pstrb  <= 4'hF;             // All bytes
        cmd_pprot  <= 3'b000;           // Normal access
    end else if (cmd_ready) begin
        cmd_valid  <= 1'b0;
    end
end
```

```
// Handle read response
always_ff @(posedge clk) begin
    if (rsp_valid && rsp_ready) begin
        if (!rsp_pslverr) begin
            status_value <= rsp_prdata;
            read_complete <= 1'b1;
        end else begin
            read_error <= 1'b1;
        end
    end
end

assign rsp_ready = 1'b1;  // Always ready to accept responses
```

3.8.4 Burst Operation Example

```
// Multiple register access sequence
parameter int NUM_REGS = 8;
logic [31:0] reg_addresses [NUM_REGS] = '{
    32'h1000_0000, 32'h1000_0004, 32'h1000_0008, 32'h1000_000C,
    32'h1000_0010, 32'h1000_0014, 32'h1000_0018, 32'h1000_001C
};
logic [31:0] reg_data [NUM_REGS];
int reg_index;

// Issue multiple commands
always_ff @(posedge clk) begin
    if (start_burst && reg_index < NUM_REGS) begin
        if (cmd_ready || !cmd_valid) begin
            cmd_valid  <= 1'b1;
            cmd_pwrite <= 1'b1;
            cmd_paddr  <= reg_addresses[reg_index];
            cmd_pwdata <= reg_data[reg_index];
            cmd_pstrb  <= 4'hF;
            cmd_pprot  <= 3'b000;
            reg_index  <= reg_index + 1;
        end
    end else begin
        cmd_valid <= 1'b0;
    end
end
```

3.9 Integration with APB Interconnect

3.9.1 APB Crossbar Integration

```
// APB system with crossbar
module apb_system (
```

```
    input logic clk, resetn,
    // CPU interface
    input logic [31:0] cpu_addr,
    input logic [31:0] cpu_wdata,
    input logic        cpu_write,
    input logic        cpu_valid,
    output logic       cpu_ready,
    output logic [31:0] cpu_rdata,
    output logic        cpu_error
);

    // APB master
    apb_master u_apb_master (
        .pclk(clk),
        .presetn(resetn),
        .m_apb_*(apb_m_*),
        // CPU command interface
        .cmd_valid(cpu_valid),
        .cmd_ready(cpu_ready),
        .cmd_pwrite(cpu_write),
        .cmd_paddr(cpu_addr),
        .cmd_pwdata(cpu_wdata),
        .cmd_pstrb(4'hF),
        .cmd_pprot(3'b000),
        // CPU response interface
        .rsp_valid(rsp_valid),
        .rsp_ready(1'b1),
        .rsp_prdata(cpu_rdata),
        .rsp_pslverr(cpu_error)
    );

    // APB crossbar to multiple slaves
    apb_xbar #(
        .NUM_SLAVES(4),
        .ADDR_WIDTH(32),
        .DATA_WIDTH(32)
    ) u_apb_xbar (
        .pclk(clk),
        .presetn(resetn),
        // Master interface
        .s_apb_*(apb_m_*),
        // Slave interfaces
        .m_apb_*(slave_apb_*)
    );

endmodule
```

3.10 Advanced Features

3.10.1 Protection Attributes (PPROT)

The module supports APB protection attributes:

PPROT[2:0]	Description
000	Normal, Non-secure, Data
001	Normal, Non-secure, Instruction
010	Normal, Secure, Data
011	Normal, Secure, Instruction
100	Privileged, Non-secure, Data
101	Privileged, Non-secure, Instruction
110	Privileged, Secure, Data
111	Privileged, Secure, Instruction

3.10.2 Write Strobe Support

Fine-grained byte-level write control:

```
// Example: Write only upper 16 bits of 32-bit register
cmd_pwdata <= {new_upper_data, 16'h0000};
cmd_pstrb  <= 4'b1100;  // Only upper 2 bytes
```

3.10.3 Error Handling

Comprehensive error handling and reporting:

```
always_ff @(posedge clk) begin
    if (rsp_valid && rsp_ready) begin
        if (rsp_pslverr) begin
            // Log error details
            error_count <= error_count + 1;
            error_addr  <= last_cmd_addr;
            error_type  <= last_cmd_write ? ERR_WRITE : ERR_READ;
        end
    end
end
```

3.11 Performance Optimization

3.11.1 FIFO Depth Selection

Choose FIFO depths based on system requirements:

Application	CMD_DEPTH	RSP_DEPTH	Rationale
CPU Access	2-4	2-4	Low latency, simple access patterns
DMA Access	6-8	6-8	Burst operations, higher throughput
Multi-Master	8-10	8-10	Multiple requestors, avoid starvation

3.11.2 Clock Gating

For power-sensitive applications, use `apb_master_cg`:

```
apb_master_cg #(
    .ADDR_WIDTH(32),
    .DATA_WIDTH(32)
) u_apb_master_cg (
    // Standard interfaces
    .*,
    // Clock gating control
    .cg_enable(apb_active),
    .cg_test_enable(scan_mode)
);
```

3.12 Synthesis Considerations

3.12.1 Area Optimization

- Reduce FIFO depths for area-constrained designs
- Use synchronous reset for smaller area
- Consider shared resources for multiple APB masters

3.12.2 Timing Optimization

- Register all outputs for timing closure
- Use appropriate FIFO depths to break critical paths
- Consider skid buffers for high-frequency operation

3.12.3 Power Optimization

- Use clock gating variant when available
- Implement power gating for inactive masters
- Use appropriate FIFO depths to minimize switching

3.13 Verification Notes

3.13.1 Protocol Compliance

- Verify APB setup and access phases
- Check PSEL and PENABLE timing
- Validate PREADY handling and wait states

3.13.2 Interface Verification

- Test command/response FIFO overflow/underflow
- Verify backpressure handling
- Check error propagation and handling

3.13.3 Performance Verification

- Measure sustained throughput
- Verify FIFO efficiency
- Check latency under various loads

3.14 Related Modules

- **apb_master_cg**: Clock-gated version for power optimization
- **apb_slave**: APB slave implementation
- **apb_xbar**: APB crossbar for multi-slave systems
- **apb_monitor**: APB protocol monitor and analyzer

The `apb_master` module provides a complete, high-performance solution for APB master functionality with advanced buffering, full protocol compliance, and comprehensive system integration capabilities.

3.15 Navigation

- ← Back to RTLAmba Index
- ← Back to Main Documentation Index

4 apb_master_stub

A lightweight APB master stub module that provides packed command/response interfaces for simplified testbench integration and system-level testing.

4.1 Overview

The `apb_master_stub` module acts as a simple APB master that accepts packed command packets and returns packed response packets. It's designed for testbench use where a simple APB master is needed without the full functionality of the standard `apb_master` module. The packed interface simplifies integration with test drivers and verification components.

4.2 Module Declaration

```
module apb_master_stub #(
    parameter int CMD_DEPTH         = 6,
    parameter int RSP_DEPTH         = 6,
    parameter int DATA_WIDTH        = 32,
    parameter int ADDR_WIDTH        = 32,
    parameter int STRB_WIDTH        = DATA_WIDTH / 8,
    parameter int CMD_PACKET_WIDTH  = ADDR_WIDTH + DATA_WIDTH +
STRB_WIDTH + 3 + 1 + 1 + 1,
                                      // addr, data, strb, prot,
pwrite, first, last
    parameter int RESP_PACKET_WIDTH = DATA_WIDTH + 1 + 1 + 1, // data,
pslverr, first, last
    // Short Parameters
    parameter int DW  = DATA_WIDTH,
    parameter int AW  = ADDR_WIDTH,
    parameter int SW  = STRB_WIDTH,
    parameter int CPW = CMD_PACKET_WIDTH,
    parameter int RPW = RESP_PACKET_WIDTH
) (
    // Clock and Reset
    input  logic                        pclk,
    input  logic                        presetn,

    // APB Master Interface
    output logic                        m_apb_PSEL,
    output logic                        m_apb_PENABLE,
    output logic [ADDR_WIDTH-1:0]       m_apb_PADDR,
```

```
    output logic                          m_apb_PWRITE,
    output logic [DATA_WIDTH-1:0]         m_apb_PWDATA,
    output logic [STRB_WIDTH-1:0]         m_apb_PSTRB,
    output logic [2:0]                    m_apb_PPROT,
    input  logic [DATA_WIDTH-1:0]         m_apb_PRDATA,
    input  logic                          m_apb_PSLVERR,
    input  logic                          m_apb_PREADY,

    // Command Packet Interface (packed)
    input  logic                          cmd_valid,
    output logic                          cmd_ready,
    input  logic [CMD_PACKET_WIDTH-1:0]   cmd_data,

    // Response Packet Interface (packed)
    output logic                          rsp_valid,
    input  logic                          rsp_ready,
    output logic [RESP_PACKET_WIDTH-1:0] rsp_data
);
```

4.3 Parameters

Parameter	Type	Default	Description
CMD_DEPTH	int	6	Command FIFO depth (log2)
RSP_DEPTH	int	6	Response FIFO depth (log2)
DATA_WIDTH	int	32	APB data bus width
ADDR_WIDTH	int	32	APB address bus width
STRB_WIDTH	int	DATA_WIDTH/8	Write strobe width (calculated)
CMD_PACKET_WIDTH	int	Calculated	Command packet width
RESP_PACKET_WIDTH	int	Calculated	Response packet width

4.4 Ports

4.4.1 Clock and Reset

Port	Width	Direction	Description
pclk	1	Input	APB clock
presetn	1	Input	APB active-low reset

4.4.2 APB Master Interface

Port	Width	Direction	Description
m_apb_PSEL	1	Output	Peripheral select
m_apb_PENABLE	1	Output	Enable signal
m_apb_PADDR	AW	Output	Address bus
m_apb_PWRITE	1	Output	Write/read (1=write, 0=read)
m_apb_PWDATA	DW	Output	Write data
m_apb_PSTRB	SW	Output	Write strobe
m_apb_PPROT	3	Output	Protection attributes
m_apb_PRDATA	DW	Input	Read data
m_apb_PSLVERR	1	Input	Slave error
m_apb_PREADY	1	Input	Ready signal

4.4.3 Packed Command Interface

Port	Width	Direction	Description
cmd_valid	1	Input	Command packet valid
cmd_ready	1	Output	Ready for command packet

Port	Width	Direction	Description
cmd_data	CPW	Input	Packed command (addr, data, strb, prot, pwrite, first, last)

4.4.4 Packed Response Interface

Port	Width	Direction	Description
rsp_valid	1	Output	Response packet valid
rsp_ready	1	Input	Ready for response packet
rsp_data	RPW	Output	Packed response (data, pslverr, first, last)

4.5 Functional Description

4.5.1 Packet Interface

The stub uses packed interfaces to simplify testbench integration:

Command Packet Format (MSB to LSB): - `last` (1 bit): Last transfer indicator - `first` (1 bit): First transfer indicator - `pwrite` (1 bit): Write/read operation - `pprot` (3 bits): Protection attributes - `pstrb` (SW bits): Write strobe - `pwdata` (DW bits): Write data - `paddr` (AW bits): Address

Response Packet Format (MSB to LSB): - `last` (1 bit): Last transfer indicator - `first` (1 bit): First transfer indicator - `pslverr` (1 bit): Slave error - `prdata` (DW bits): Read data

4.5.2 Operation

1. Test driver presents packed command on `cmd_data` with `cmd_valid=1`
2. Stub accepts when `cmd_ready=1`
3. Stub unpacks command and drives APB protocol
4. On APB completion, stub packs response and asserts `rsp_valid=1`
5. Test driver reads response when `rsp_ready=1`

4.6 Usage Example

```
// Instantiate APB master stub
apb_master_stub #(
    .DATA_WIDTH(32),
    .ADDR_WIDTH(16)
) u_apb_master_stub (
    .pclk(clk),
    .presetn(rst_n),
    // APB interface to slave/interconnect
```

```
    .m_apb_PSEL(apb_psel),
    .m_apb_PENABLE(apb_penable),
    .m_apb_PADDR(apb_paddr),
    .m_apb_PWRITE(apb_pwrite),
    .m_apb_PWDATA(apb_pwdata),
    .m_apb_PSTRB(apb_pstrb),
    .m_apb_PPROT(apb_pprot),
    .m_apb_PRDATA(apb_prdata),
    .m_apb_PSLVERR(apb_pslverr),
    .m_apb_PREADY(apb_pready),
    // Packed interface to test driver
    .cmd_valid(test_cmd_valid),
    .cmd_ready(test_cmd_ready),
    .cmd_data(test_cmd_data),
    .rsp_valid(test_rsp_valid),
    .rsp_ready(test_rsp_ready),
    .rsp_data(test_rsp_data)
);

// Example: Send write command (in testbench)
// Pack command: addr=0x1000, data=0xDEADBEEF, write=1
assign test_cmd_data = {1'b1, 1'b1, 1'b1, 3'b000, 4'hF, 32'hDEADBEEF,
16'h1000};
assign test_cmd_valid = 1'b1;
```

4.7 Design Notes

4.7.1 Testbench Usage

This module is intended for: - System-level testbenches - Integration testing - Simple APB traffic generation - Verification component integration

For production use, consider the full-featured `apb_master` module.

4.7.2 Packed Interface Benefits

- Simplified connection to test drivers
- Reduces signal count
- Easier integration with verification IP
- Convenient for parameterized test sequences

4.8 Related Modules

- `apb_master.sv` - Full-featured APB master with independent interfaces
- `apb_slave_stub.sv` - Companion APB slave stub
- `apb_monitor.sv` - APB transaction monitoring

4.9 References

- **APB Protocol**: AMBA APB Protocol Specification v2.0
- **Full APB Master**: apb_master.md
- **APB Slave Stub**: apb_slave_stub.md

4.10 Navigation

- **← Back to APB Index** (if exists, otherwise remove this line)
- **← Back to RTLAmba Index**
- **← Back to Main Documentation Index**

5 APB Slave with CDC (Clock-Gated)

Module: `apb_slave_cdc_cg.sv` **Base Module:** apb_slave_cdc **Location:** `rtl/amba/apb/`
Status: √ Production Ready

5.1 Quick Reference

This is the **clock-gated variant** of apb_slave_cdc.

For complete clock-gating documentation, usage examples, and configuration guidelines, see:

→ **Clock-Gated Variants Guide**

5.2 Summary

The `apb_slave_cdc_cg` module adds power optimization to `apb_slave_cdc` through activity-based clock gating:

- √ **Same Functionality:** 100% equivalent to base module
- √ **Power Savings:** 25-70% depending on traffic utilization
- √ **Configurable:** Idle threshold, gating domains, enable/disable
- √ **Zero Overhead When Disabled:** `ENABLE_CLOCK_GATING=0` → identical to base

5.3 Common Parameters

In addition to all apb_slave_cdc parameters:

Parameter	Default	Description
ENABLE_CLOCK_GATING	1	Master enable (0=disable, identical to base)
CG_IDLE_CYCLES	8	Cycles before clock gating activates
CG_GATE_*	1	Domain-specific gating enables

5.4 Quick Usage

```
apb_slave_cdc_cg #(
    // Base module parameters (see apb_slave_cdc.md)
    .AXI_ID_WIDTH(8),
    .AXI_ADDR_WIDTH(32),
    .AXI_DATA_WIDTH(64),

    // Clock gating (see CG guide for details)
    .ENABLE_CLOCK_GATING(1),
    .CG_IDLE_CYCLES(8)
) u_cg (
    .aclk(clk),
    .aresetn(rst_n),
    // ... all other ports same as apb_slave_cdc
);
```

5.5 Documentation

- **Base Module Functionality:** apb_slave_cdc.md
- **Clock Gating Guide:** clock_gated_variants.md
- **Detailed CG Examples:**
  - axi4_master_rd_mon_cg.md (AXI4 monitor)
  - axil4_master_rd_mon_cg.md (AXIL4 monitor)
  - apb_slave_cg.md (APB interface)

5.6 Navigation

- ← Back to APB Index
- ← Back to RTLAmba Index
- ← Back to Main Documentation Index

6 APB Slave CDC

Module: `apb_slave_cdc.sv` **Location:** `rtl/amba/apb/` **Status:** √ Production Ready

6.1 Overview

The APB Slave CDC (Clock Domain Crossing) module provides a complete APB slave interface with integrated clock domain crossing between the APB (pclk) domain and an AXI/GAXI (aclk) domain. This enables safe integration of APB peripherals running at different clock frequencies.

6.1.1 Key Features

- √ **Safe CDC:** Proper clock domain crossing with handshake protocol
- √ **Dual Clock Domains:** APB (pclk) and AXI (aclk) operate independently
- √ **Full APB4/APB5 Support:** Complete protocol compliance
- √ **Command/Response Interface:** Clean GAXI-style backend interface
- √ **Buffered Operation:** Integrated skid buffers for elastic storage

---

6.2 Module Interface

```
module apb_slave_cdc #(
    parameter int ADDR_WIDTH  = 32,
    parameter int DATA_WIDTH  = 32,
    parameter int STRB_WIDTH  = DATA_WIDTH / 8,
    parameter int PROT_WIDTH  = 3,
    parameter int DEPTH       = 2
) (
    // Clock and Reset
    input  logic                  aclk,
    input  logic                  aresetn,
    input  logic                  pclk,
    input  logic                  presetn,

    // APB Slave Interface (pclk domain)
    input  logic                  s_apb_PSEL,
```

```
    input  logic              s_apb_PENABLE,
    output logic              s_apb_PREADY,
    input  logic [AW-1:0]     s_apb_PADDR,
    input  logic              s_apb_PWRITE,
    input  logic [DW-1:0]     s_apb_PWDATA,
    input  logic [SW-1:0]     s_apb_PSTRB,
    input  logic [PW-1:0]     s_apb_PPROT,
    output logic [DW-1:0]     s_apb_PRDATA,
    output logic              s_apb_PSLVERR,

    // Command Interface (aclk domain)
    output logic              cmd_valid,
    input  logic              cmd_ready,
    output logic              cmd_pwrite,
    output logic [AW-1:0]     cmd_paddr,
    output logic [DW-1:0]     cmd_pwdata,
    output logic [SW-1:0]     cmd_pstrb,
    output logic [PW-1:0]     cmd_pprot,

    // Response Interface (aclk domain)
    input  logic              rsp_valid,
    output logic              rsp_ready,
    input  logic [DW-1:0]     rsp_prdata,
    input  logic              rsp_pslverr
);
```

6.3 Clock Domains

6.3.1 APB Domain (pclk)

- APB slave interface signals
- Typical frequency: 50-200 MHz
- Used by APB master/interconnect

6.3.2 AXI Domain (aclk)

- Command and response interfaces
- Can be faster or slower than pclk
- Used by backend processing logic

Note: The module handles the clock domain crossing safely using handshake-based CDC techniques.

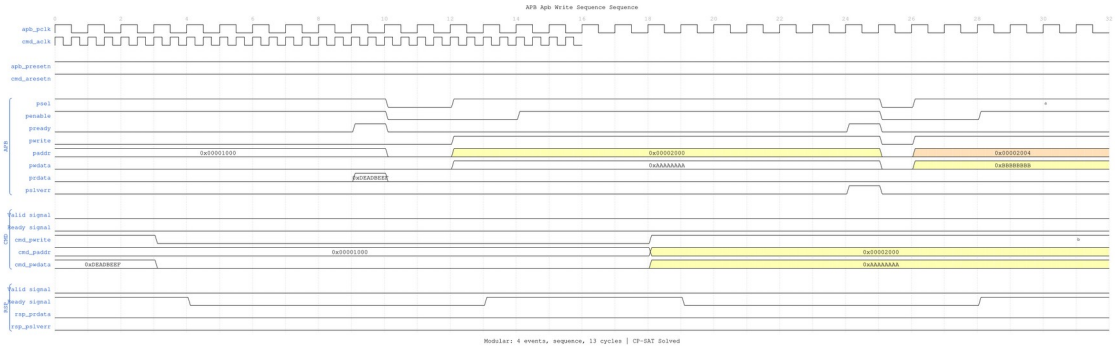
6.4 Waveforms

The following timing diagrams show CDC behavior with **both clock domains visible**:

Clock Configuration: - `apb_pclk`: 100MHz (10ns period) - `cmd_aclk`: 500MHz (2ns period), displayed with `period=0.4` for visual compactness

6.4.1 Scenario 1: Write Transaction with CDC

Shows APB write crossing from pclk domain to aclk domain:



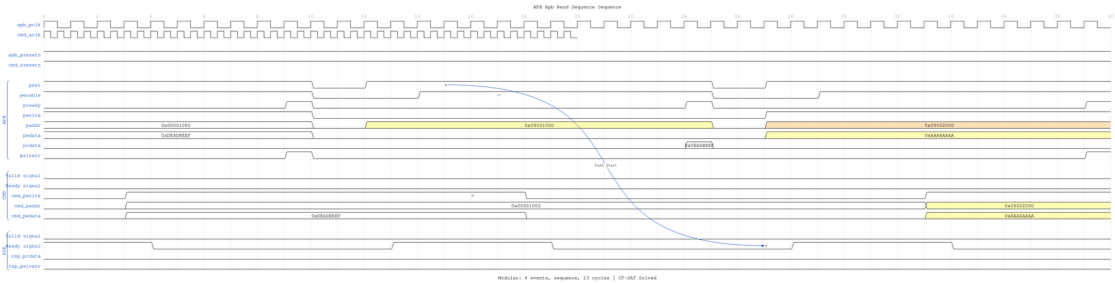
Write CDC

WaveJSON: apb_write_sequence_001.json

Key Observations: - APB transaction in pclk domain - CMD transaction crosses to aclk domain - Note the CDC latency between domains

6.4.2 Scenario 2: Read Transaction with CDC

Shows APB read with response crossing back from aclk to pclk domain:

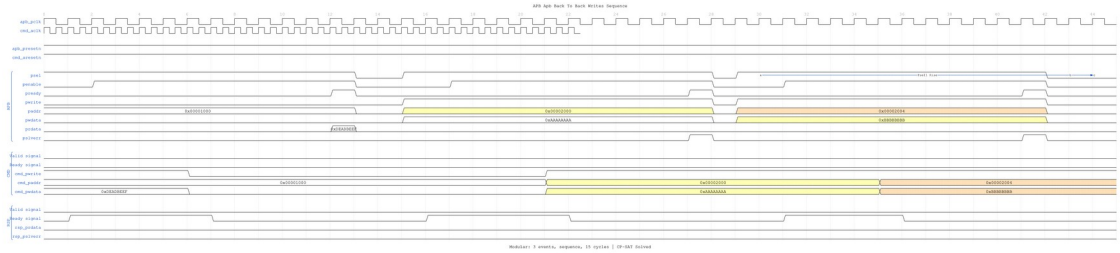


Read CDC

WaveJSON: apb_read_sequence_001.json

Key Observations: - APB read request in pclk domain - CMD crosses to aclk domain - RSP crosses back to pclk domain - Complete round-trip CDC visible

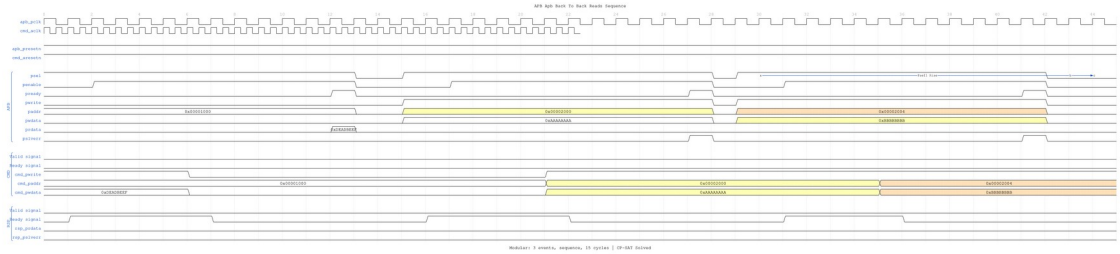
6.4.3 Scenario 3: Back-to-Back Writes with CDC



B2B Writes CDC

WaveJSON: apb_back_to_back_writes_001.json

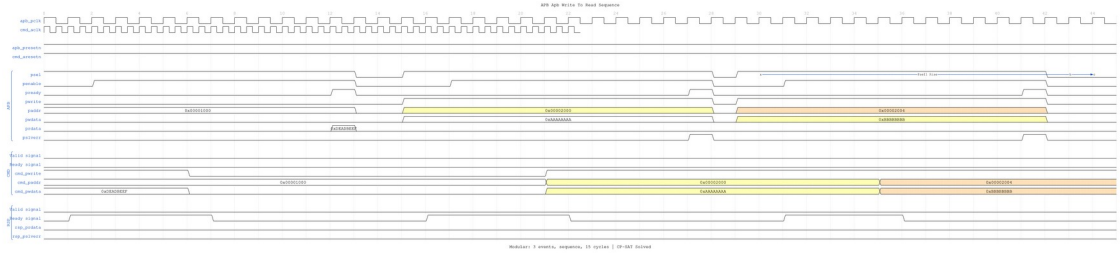
6.4.4 Scenario 4: Back-to-Back Reads with CDC



B2B Reads CDC

WaveJSON: apb_back_to_back_reads_001.json

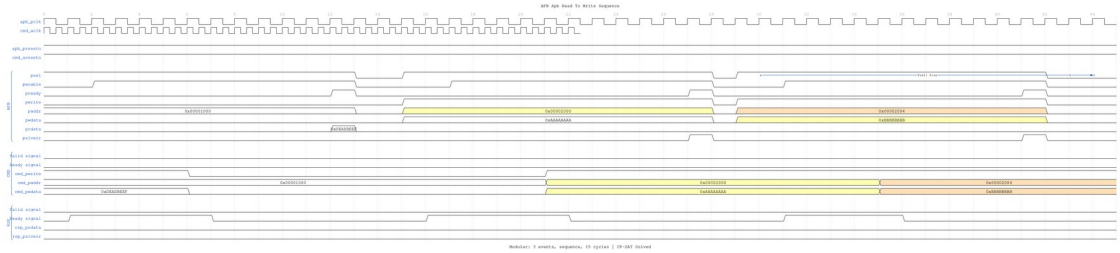
6.4.5 Scenario 5: Write-to-Read Transition with CDC



Write-to-Read CDC

WaveJSON: apb_write_to_read_001.json

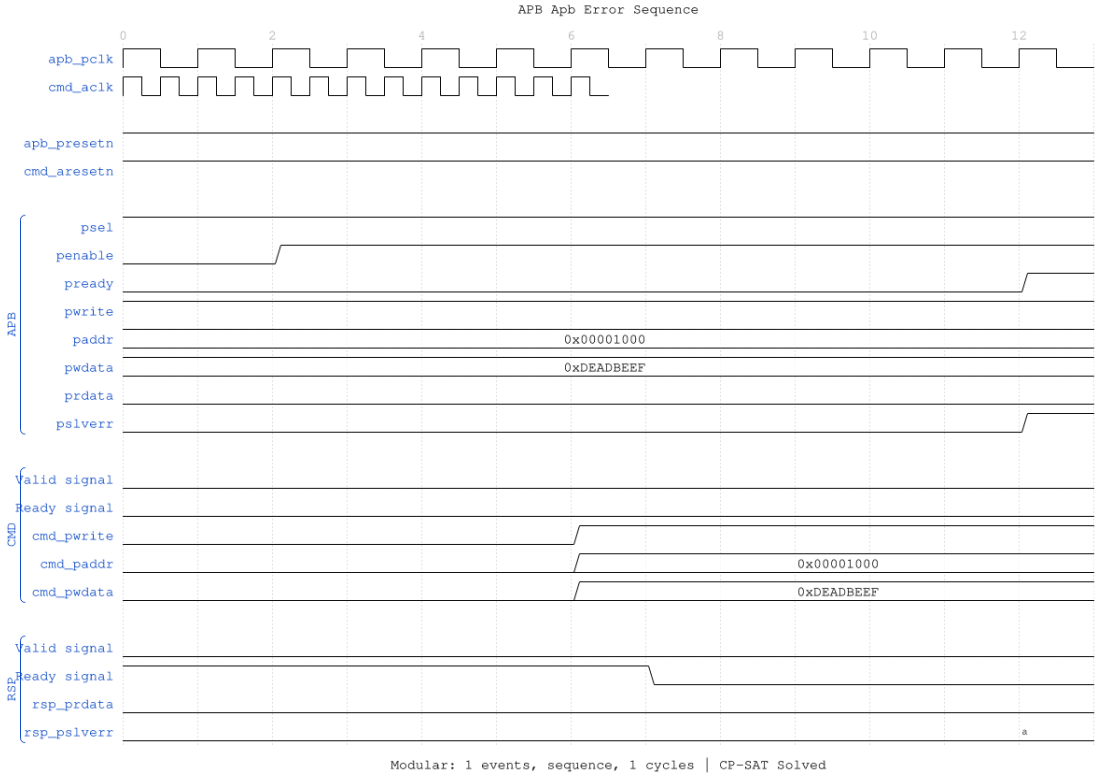
6.4.6 Scenario 6: Read-to-Write Transition with CDC



Read-to-Write CDC

WaveJSON: apb_read_to_write_001.json

6.4.7 Scenario 7: Error Response with CDC



Error CDC

WaveJSON: apb_error_001.json

6.5 Usage Example

```
apb_slave_cdc #(
    .ADDR_WIDTH(32),
    .DATA_WIDTH(32),
    .DEPTH(2)
) u_apb_cdc (
    // APB clock domain
    .pclk         (apb_clk),
    .presetn      (apb_resetn),

    // AXI clock domain
    .aclk         (axi_clk),
    .aresetn      (axi_resetn),

    // APB slave interface (pclk domain)
    .s_apb_PSEL     (apb_psel),
    .s_apb_PENABLE  (apb_penable),
    .s_apb_PREADY   (apb_pready),
    .s_apb_PADDR    (apb_paddr),
    .s_apb_PWRITE   (apb_pwrite),
    .s_apb_PWDATA   (apb_pwdata),
    .s_apb_PSTRB    (apb_pstrb),
    .s_apb_PPROT    (apb_pprot),
    .s_apb_PRDATA   (apb_prdata),
    .s_apb_PSLVERR  (apb_pslverr),

    // Command interface (aclk domain)
    .cmd_valid      (cmd_valid),
    .cmd_ready      (cmd_ready),
    .cmd_pwrite     (cmd_pwrite),
    .cmd_paddr      (cmd_paddr),
    .cmd_pwdata     (cmd_pwdata),
    .cmd_pstrb      (cmd_pstrb),
    .cmd_pprot      (cmd_pprot),

    // Response interface (aclk domain)
    .rsp_valid      (rsp_valid),
    .rsp_ready      (rsp_ready),
    .rsp_prdata     (rsp_prdata),
    .rsp_pslverr    (rsp_pslverr)
);
```

6.6 References

- **APB Slave:** apb_slave.md
- **Source:** `rtl/amba/apb/apb_slave_cdc.sv`
- **Tests:** `val/amba/test_apb_slave_cdc.py`
- **WaveDrom Test:** `val/amba/test_apb_slave_cdc.py::test_apb_slave_cdc_wavedrom`

Last Updated: 2025-10-09

6.7 Navigation

- **← Back to RTLAmba Index**
- **← Back to Main Documentation Index**

7 APB Slave Interface (Clock-Gated)

Module: `apb_slave_cg.sv` **Base Module:** apb_slave **Location:** `rtl/amba/apb/` **Status:** ✓ Production Ready

---

7.1 Overview

The `apb_slave_cg` module is a clock-gated variant of apb_slave that adds comprehensive power optimization capabilities through activity-based clock gating.

7.1.1 Key Differences from Base Module

- ✓ **Activity-Based Clock Gating:** Automatically gates clocks when subsystems are idle
- ✓ **Configurable Policies:** Fine-grained control over what gets gated and when
- ✓ **Power Monitoring:** Built-in statistics for clock gating effectiveness
- ✓ **Independent Gating Domains:** Separate control for different functional blocks
- ✓ **Zero Functional Impact:** Maintains 100% functional equivalence with base module

All other functionality is identical to the base module. See apb_slave.md for complete functional specification.

7.2 When to Use Clock-Gated Variant

Use `apb_slave_cg` when: - Power consumption is a critical concern - Design has periods of inactivity (burst traffic patterns) - FPGA/ASIC has integrated clock gating support - Meeting power budgets for battery-operated systems

Use base module (`apb_slave`) when: - Maximum performance with no power constraints - Continuous high-activity traffic - Simpler design with fewer configuration parameters - Minimizing gate count is priority

---

7.3 Clock Gating Parameters

In addition to all parameters from apb_slave, this module adds:

Parameter	Type	Default	Description
`ENABLE_CLOCK_GATING`	bit	1	Master enable for clock gating (0=disable all gating)
`CG_IDLE_CYCLES`	int	8	Number of idle cycles before asserting clock gate
`CG_GATE_DATAPATH`	bit	1	Enable clock gating for data path logic
`CG_GATE_CONTROL`	bit	1	Enable clock gating for control path logic

7.3.1 Parameter Relationships

- **`ENABLE_CLOCK_GATING = 0`**: Disables all clock gating, module behaves identically to base
- **`CG_IDLE_CYCLES`**: Higher values = more power savings but slower wake-up from idle
- **Individual `CG_GATE_*` signals**: Allow fine-grained control over which subsystems are gated

7.4 Usage Examples

7.4.1 Example 1: Maximum Power Savings (Burst Traffic)

```
apb_slave_cg #(
    // Base parameters (see apb_slave.md)
    .AXI_ID_WIDTH(8),
    .AXI_ADDR_WIDTH(32),
    .AXI_DATA_WIDTH(64),

    // Clock gating - aggressive power savings
    .ENABLE_CLOCK_GATING(1),
    .CG_IDLE_CYCLES(4),      // Gate quickly after 4 idle cycles
    .CG_GATE_DATAPATH(1),    // Gate data path
    .CG_GATE_CONTROL(1)      // Gate control path
) u_cg (
    .aclk(clk),
    .aresetn(rst_n),
    // ... connect signals same as base module
);
```

7.4.2 Example 2: Balanced Performance and Power

```
apb_slave_cg #(
    // Base parameters
    .AXI_ID_WIDTH(8),
    .AXI_ADDR_WIDTH(32),
    .AXI_DATA_WIDTH(64),

    // Clock gating - balanced approach
    .ENABLE_CLOCK_GATING(1),
    .CG_IDLE_CYCLES(16),     // Wait longer before gating (faster
wake-up)
    .CG_GATE_DATAPATH(1),    // Gate data path
    .CG_GATE_CONTROL(0)      // Keep control path active
) u_cg (
    .aclk(clk),
    .aresetn(rst_n),
    // ... connect signals same as base module
);
```

7.4.3 Example 3: Clock Gating Disabled (Functional Verification)

```
apb_slave_cg #(
    // Base parameters
    .AXI_ID_WIDTH(8),
    .AXI_ADDR_WIDTH(32),
```

```
    .AXI_DATA_WIDTH(64),

    // Clock gating - DISABLED for verification
    .ENABLE_CLOCK_GATING(0)  // Disable all gating
) u_cg (
    .aclk(clk),
    .aresetn(rst_n),
    // ... connect signals same as base module
);
```

Note: With `ENABLE_CLOCK_GATING=0`, this module is functionally identical to the base module.

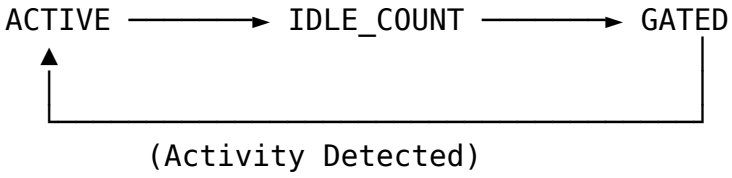
7.5 Clock Gating Architecture

7.5.1 Gating Domains

The module implements independent clock gating for these functional blocks:

1. **Data Path Domain**
   - Data buffering and forwarding logic
   - Gated when: No valid data for `CG_IDLE_CYCLES`
   - Controlled by: `CG_GATE_DATAPATH`
2. **Control Path Domain**
   - Handshake and control signal logic
   - Gated when: Interface idle for `CG_IDLE_CYCLES`
   - Controlled by: `CG_GATE_CONTROL`

7.5.2 Gating State Machine




```
States:
- ACTIVE: Clocks enabled, monitoring activity
- IDLE_COUNT: Counting CG_IDLE_CYCLES before gating
- GATED: Clocks disabled, waiting for activity
```

7.5.3 Wake-Up Latency

Configuration	Wake-Up Time	Use Case
`CG_IDLE_CYCLES=4`	~4 clock cycles	Low-latency, frequent bursts
`CG_IDLE_CYCLES=8`	~8 clock cycles	Balanced (default)
`CG_IDLE_CYCLES=16`	~16 clock cycles	Maximum power savings, infrequent traffic

7.6 Power Savings Analysis

7.6.1 Typical Power Reduction

Based on representative workloads:

Traffic Pattern	Clock Gating Enabled	Power Savings
10% Utilization	Aggressive (`CG_IDLE_CYCLES=4`)	60-70%
25% Utilization	Balanced (`CG_IDLE_CYCLES=8`)	45-55%
50% Utilization	Conservative (`CG_IDLE_CYCLES=16`)	25-35%
90% Utilization	Any configuration	5-10%

Note: Actual savings depend on FPGA/ASIC technology, tool implementation, and traffic patterns.

7.6.2 Power Monitoring Signals

The module provides these status signals for power analysis:

Signal	Width	Description
`cg_monitor_gated`	1	Monitor domain clock is gated
`cg_reporter_gated`	1	Reporter domain clock is gated

7.7 Verification Considerations

7.7.1 Clock Gating in Simulation

Recommendation: Disable clock gating during functional verification:

```
// Testbench instantiation
apb_slave_cg #(
    .ENABLE_CLOCK_GATING(0)  // Disable for faster simulation
) dut (
    // ... connections
);
```

Rationale: - Simpler waveforms (no clock gating events) - Faster simulation (no gating overhead) - Easier debug (no timing dependencies)

7.7.2 Power Analysis Verification

For power-specific verification:

1. **Enable clock gating** with realistic parameters
2. **Monitor gating signals** (`cg_*_gated`) to verify expected behavior
3. **Vary traffic patterns** to test gating effectiveness
4. **Check wake-up timing** meets system requirements

7.8 Synthesis Considerations

7.8.1 FPGA Implementations

Xilinx: - Use `ENABLE_CLOCK_GATING=1` with `BUFGCE` primitives - Tool will infer clock enables automatically - Verify with post-synthesis power analysis

Intel (Altera): - Use `ENABLE_CLOCK_GATING=1` with `ALTCLKCTRL` - May need vendor-specific clock gating primitives - Check power reports for gating effectiveness

Lattice: - Basic clock gating supported - May require manual instantiation of clock enables - Verify functionality in timing simulation

7.8.2 ASIC Implementations

- Work with foundry to select appropriate clock gating cells
- Integrated Clock Gating (ICG) cells provide best results
- Consider hold-time implications of clock gating
- Verify power intent with UPF (Unified Power Format)

7.9 Related Modules

- **apb_slave** - Base module (non-clock-gated)

7.10 See Also

- **Power Optimization Guide:** `docs/POWER_OPTIMIZATION_GUIDE.md`
- **Clock Gating Best Practices:** `docs/CLOCK_GATING_GUIDE.md`
- **AMBA Subsystem Overview:** `docs/markdown/RTLAmba/overview.md`

7.11 Navigation

- ← Back to Base Module
- ← Back to RTLAmba Index
- ← Back to Main Documentation Index

8 apb_slave

An Advanced Peripheral Bus (APB) slave module that provides a complete APB4/APB5 compliant interface with buffered command/response processing for high-performance peripheral integration.

8.1 Overview

The `apb_slave` module implements a full APB slave interface with integrated command and response buffering using GAXI skid buffers. It provides seamless protocol translation between the APB interface and a simple command/response interface, enabling easy integration of custom peripherals and register blocks into APB-based systems.

8.2 Module Declaration

```
module apb_slave #(
    parameter int ADDR_WIDTH      = 32,
    parameter int DATA_WIDTH      = 32,
    parameter int STRB_WIDTH      = DATA_WIDTH / 8,
    parameter int PROT_WIDTH      = 3,
    parameter int DEPTH           = 2,
    // Short Parameters
    parameter int AW  = ADDR_WIDTH,
```

```
    parameter int DW  = DATA_WIDTH,
    parameter int SW  = STRB_WIDTH,
    parameter int PW  = PROT_WIDTH,
    parameter int CPW = AW + DW + SW + PW + 1,  // Command packet
width
    parameter int RPW = DW + 1                  // Response packet
width
) (
    // Clock and Reset
    input  logic              pclk,
    input  logic              presetn,

    // APB Slave Interface
    input  logic              s_apb_PSEL,
    input  logic              s_apb_PENABLE,
    output logic              s_apb_PREADY,
    input  logic [AW-1:0]     s_apb_PADDR,
    input  logic              s_apb_PWRITE,
    input  logic [DW-1:0]     s_apb_PWDATA,
    input  logic [SW-1:0]     s_apb_PSTRB,
    input  logic [PW-1:0]     s_apb_PPROT,
    output logic [DW-1:0]     s_apb_PRDATA,
    output logic              s_apb_PSLVERR,

    // Command Interface
    output logic              cmd_valid,
    input  logic              cmd_ready,
    output logic              cmd_pwrite,
    output logic [AW-1:0]     cmd_paddr,
    output logic [DW-1:0]     cmd_pwdata,
    output logic [SW-1:0]     cmd_pstrb,
    output logic [PW-1:0]     cmd_pprot,

    // Response Interface
    input  logic              rsp_valid,
    output logic              rsp_ready,
    input  logic [DW-1:0]     rsp_prdata,
    input  logic              rsp_pslverr
);
```

8.3 Parameters

Parameter	Type	Default	Description
ADDR_WIDTH	int	32	APB address bus width
DATA_WIDTH	int	32	APB data bus

Parameter	Type	Default	Description
			width
STRB_WIDTH	int	DATA_WIDTH/8	Write strobe width (calculated)
PROT_WIDTH	int	3	APB protection signal width
DEPTH	int	2	Skid buffer depth (2^DEPTH entries)

8.4 Ports

8.4.1 Clock and Reset

Port	Width	Direction	Description
pclk	1	Input	APB clock
presetn	1	Input	APB active-low reset

8.4.2 APB Slave Interface

Port	Width	Direction	Description
s_apb_PSEL	1	Input	APB select signal
s_apb_PENABLE	1	Input	APB enable signal
s_apb_PREADY	1	Output	APB ready signal
s_apb_PADDR	ADDR_WIDTH	Input	APB address
s_apb_PWRITE	1	Input	APB write/read indicator
s_apb_PWDATA	DATA_WIDTH	Input	APB write data
s_apb_PSTRB	STRB_WIDTH	Input	APB write strobes

Port	Width	Direction	Description
s_apb_PPROT	PROT_WIDTH	Input	APB protection attributes
s_apb_PRDATA	DATA_WIDTH	Output	APB read data
s_apb_PSLVERR	1	Output	APB slave error

8.4.3 Command Interface

Port	Width	Direction	Description
cmd_valid	1	Output	Command valid
cmd_ready	1	Input	Command ready (from backend)
cmd_pwrite	1	Output	Command write/read
cmd_paddr	ADDR_WIDTH	Output	Command address
cmd_pwdata	DATA_WIDTH	Output	Command write data
cmd_pstrb	STRB_WIDTH	Output	Command write strobes
cmd_pprot	PROT_WIDTH	Output	Command protection attributes

8.4.4 Response Interface

Port	Width	Direction	Description
rsp_valid	1	Input	Response valid
rsp_ready	1	Output	Response ready
rsp_prdata	DATA_WIDTH	Input	Response read data
rsp_pslverr	1	Input	Response error status

8.5 Functionality

8.5.1 APB Protocol Implementation

The module implements the complete APB slave protocol with a three-state finite state machine:

1. **IDLE**: Waiting for APB transaction (PSEL && PENABLE)
2. **BUSY**: Processing transaction, waiting for backend response
3. **WAIT**: Completing transaction, returning to IDLE

8.5.2 Command/Response Flow

```
APB Master → APB Slave Interface → Command Buffer → Command Interface
→ Backend
                    ↓                          ↓              ↓
APB Master ← APB Slave Interface ← Response Buffer ← Response
Interface ← Backend
```

8.5.3 Buffering Architecture

The module uses two GAXI skid buffers for decoupling:

- **Command Buffer**: Stores incoming APB transactions
- **Response Buffer**: Stores backend responses for APB return

8.5.4 Key Features

- **APB4/APB5 Compliance**: Full protocol support including PSTRB and PPROT
- **Buffered Operation**: Command and response skid buffers prevent blocking
- **Flow Control**: Proper ready/valid handshaking on all interfaces
- **Error Handling**: PSLVERR propagation from backend to APB
- **Zero Wait State**: Can achieve single-cycle response when buffers are primed

8.6 State Machine Operation

8.6.1 State Transitions

```
IDLE → BUSY: s_apb_PSEL && s_apb_PENABLE && r_cmd_ready
BUSY → WAIT: r_rsp_valid (response available)
WAIT → IDLE: Automatic (1 cycle completion)
```

8.6.2 APB Transaction Timing

State	APB Signals	Internal Operation
IDLE	PREADY=0	Wait for PSEL && PENABLE
BUSY	PREADY=0	Issue command, wait for response
WAIT	PREADY=1, PRDATA/PSLVERR valid	Complete transaction

8.7 Timing Characteristics

8.7.1 Transaction Latency

Characteristic	Value	Description
Minimum Latency	2 clock cycles	Command → Response (buffered)
Buffer Latency	1 clock cycle	Command/Response buffering
APB Setup	1 clock cycle	PSEL to PENABLE
Response Processing	1+ clock cycles	Backend processing time

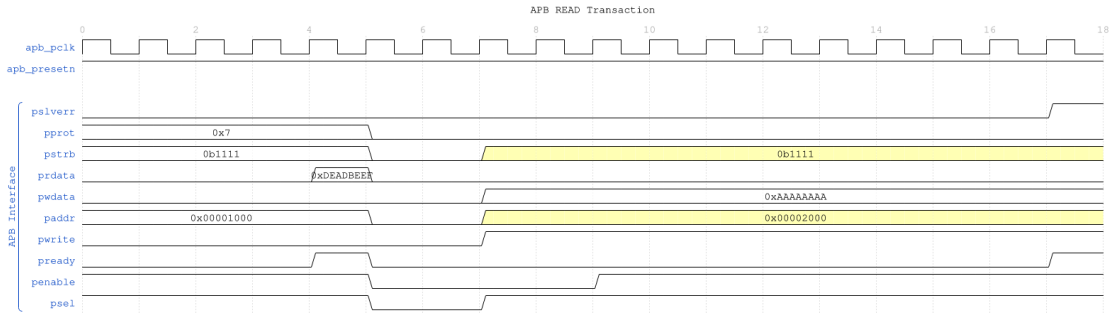
8.7.2 Performance Metrics

Metric	Value	Conditions
Maximum Frequency	200-400 MHz	Technology dependent
Peak Throughput	1.6-3.2 GB/s	32-bit data, continuous access
Buffer Depth	4 entries	With default DEPTH=2
Concurrent Transactions	Up to buffer depth	Limited by backend capacity

8.8 Waveforms

The following timing diagrams show the comprehensive APB slave behavior across 7 scenarios:

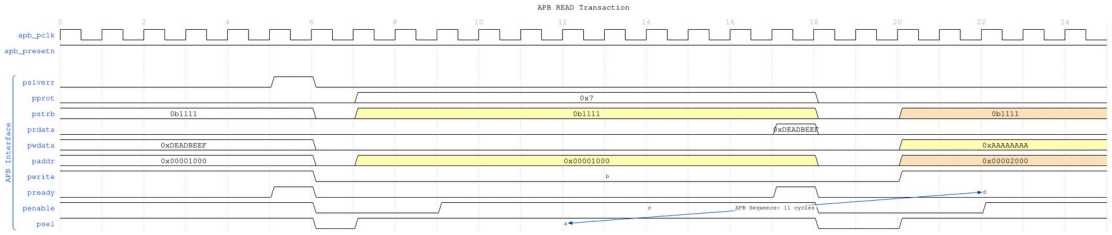
8.8.1 Scenario 1: Basic Write Transaction



APB Write

WaveJSON: apb_write_sequence_001.json

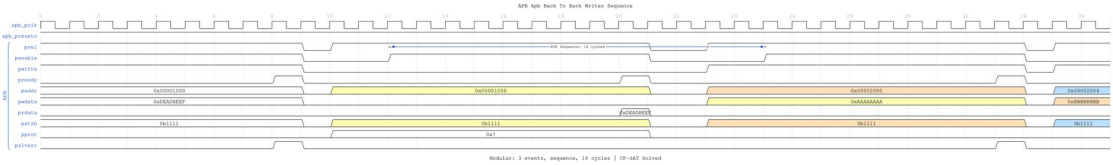
8.8.2 Scenario 2: Basic Read Transaction



APB Read

WaveJSON: apb_read_sequence_001.json

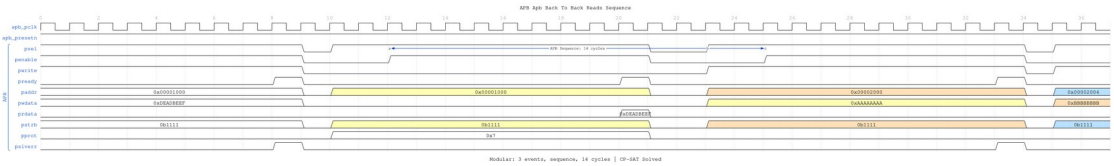
8.8.3 Scenario 3: Back-to-Back Writes



B2B Writes

WaveJSON: apb_back_to_back_writes_001.json

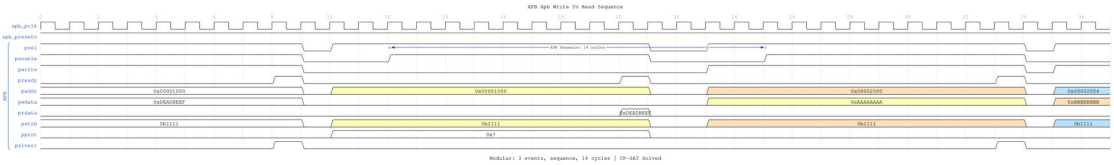
8.8.4 Scenario 4: Back-to-Back Reads



B2B Reads

WaveJSON: apb_back_to_back_reads_001.json

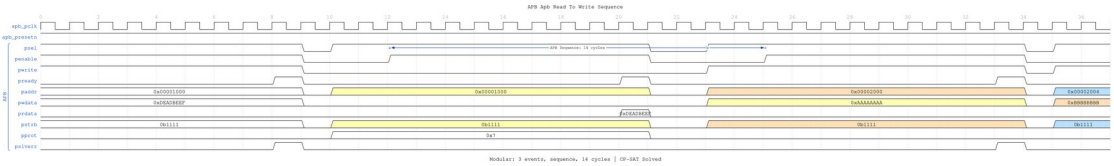
8.8.5 Scenario 5: Write-to-Read Transition



Write-to-Read

WaveJSON: apb_write_to_read_001.json

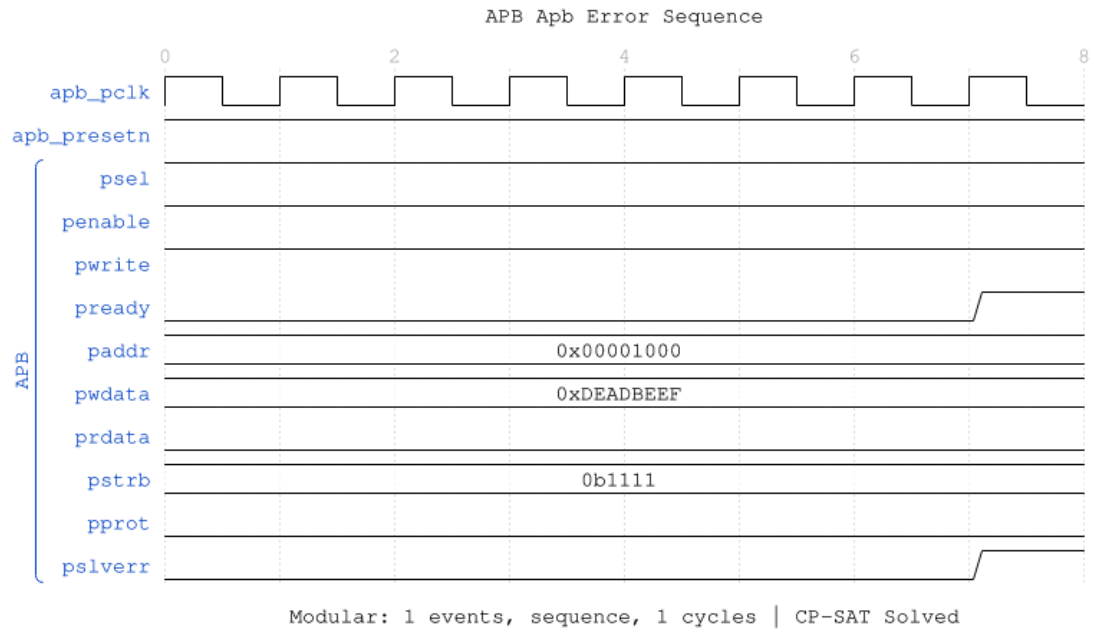
8.8.6 Scenario 6: Read-to-Write Transition



Read-to-Write

WaveJSON: apb_read_to_write_001.json

8.8.7 Scenario 7: Error Response



Error

WaveJSON: apb_error_001.json

8.9 Usage Examples

8.9.1 Basic Register Block Interface

```
apb_slave #(
    .ADDR_WIDTH(16),     // 64KB address space
    .DATA_WIDTH(32),
    .DEPTH(2)            // 4-entry buffers
) u_reg_slave (
    .pclk         (apb_clk),
    .presetn      (apb_resetn),

    // APB interface from master
    .s_apb_PSEL     (apb_psel),
    .s_apb_PENABLE  (apb_penable),
    .s_apb_PREADY   (apb_pready),
    .s_apb_PADDR    (apb_paddr),
    .s_apb_PWRITE   (apb_pwrite),
    .s_apb_PWDATA   (apb_pwdata),
    .s_apb_PSTRB    (apb_pstrb),
```

```
    .s_apb_PPROT    (apb_pprot),
    .s_apb_PRDATA   (apb_prdata),
    .s_apb_PSLVERR  (apb_pslverr),

    // Command interface to register block
    .cmd_valid      (reg_cmd_valid),
    .cmd_ready      (reg_cmd_ready),
    .cmd_pwrite     (reg_write),
    .cmd_paddr      (reg_addr),
    .cmd_pwdata     (reg_wdata),
    .cmd_pstrb      (reg_strb),
    .cmd_pprot      (reg_prot),

    // Response interface from register block
    .rsp_valid      (reg_rsp_valid),
    .rsp_ready      (reg_rsp_ready),
    .rsp_prdata     (reg_rdata),
    .rsp_pslverr    (reg_error)
);
```

8.9.2 Register Block Backend Implementation

```
// Simple register block backend
module register_block (
    input  logic        clk,
    input  logic        resetn,

    // Command interface
    input  logic        cmd_valid,
    output logic        cmd_ready,
    input  logic        cmd_write,
    input  logic [15:0] cmd_addr,
    input  logic [31:0] cmd_wdata,
    input  logic [3:0]  cmd_strb,

    // Response interface
    output logic        rsp_valid,
    input  logic        rsp_ready,
    output logic [31:0] rsp_rdata,
    output logic        rsp_error
);

    // Register array
    logic [31:0] registers [16];

    // Command processing
    always_ff @(posedge clk) begin
```

```
        if (cmd_valid && cmd_ready) begin
            if (cmd_write) begin
                // Write operation with byte strobes
                for (int i = 0; i < 4; i++) begin
                    if (cmd_strb[i]) begin
                        registers[cmd_addr[5:2]][i*8+:8] <=
cmd_wdata[i*8+:8];
                    end
                end
            end
        end
    end

    // Response generation
    always_ff @(posedge clk) begin
        if (!resetn) begin
            rsp_valid <= 1'b0;
            rsp_rdata <= 32'h0;
            rsp_error <= 1'b0;
        end else if (cmd_valid && cmd_ready) begin
            rsp_valid <= 1'b1;
            rsp_rdata <= cmd_write ? 32'h0 : registers[cmd_addr[5:2]];
            rsp_error <= (cmd_addr >= 16*4); // Address range check
        end else if (rsp_ready) begin
            rsp_valid <= 1'b0;
        end
    end

    assign cmd_ready = !rsp_valid || rsp_ready;

endmodule
```

8.9.3 Memory Interface Example

```
// APB slave for memory interface
apb_slave #(
    .ADDR_WIDTH(20),     // 1MB address space
    .DATA_WIDTH(32),
    .DEPTH(3)            // 8-entry buffers for memory latency
) u_mem_slave (
    .pclk         (mem_clk),
    .presetn      (mem_resetn),

    // APB interface
    .s_apb_*(apb_*),

    // Memory command interface
    .cmd_valid    (mem_cmd_valid),
```

```
    .cmd_ready    (mem_cmd_ready),
    .cmd_pwrite   (mem_write),
    .cmd_paddr    (mem_addr),
    .cmd_pwdata   (mem_wdata),
    .cmd_pstrb    (mem_strb),
    .cmd_pprot    (mem_prot),

    // Memory response interface
    .rsp_valid    (mem_rsp_valid),
    .rsp_ready    (mem_rsp_ready),
    .rsp_prdata   (mem_rdata),
    .rsp_pslverr  (mem_error)
);

// Memory controller interface
memory_controller u_mem_ctrl (
    .clk          (mem_clk),
    .resetn       (mem_resetn),

    // APB slave interface
    .cmd_*(mem_cmd_*),
    .rsp_*(mem_rsp_*),

    // Physical memory interface
    .mem_*(ddr_*)
);
```

8.10 Advanced Integration Patterns

8.10.1 Multi-Register Bank System

```
module multi_register_system (
    input logic clk, resetn,

    // APB interface
    input  logic        apb_psel,
    input  logic        apb_penable,
    output logic        apb_pready,
    input  logic [31:0] apb_paddr,
    input  logic        apb_pwrite,
    input  logic [31:0] apb_pwdata,
    input  logic [3:0]  apb_pstrb,
    input  logic [2:0]  apb_pprot,
    output logic [31:0] apb_prdata,
    output logic        apb_pslverr
);
```

```
    // APB slave with address-based routing
    apb_slave u_apb_slave (
        .pclk(clk),
        .presetn(resetn),
        .s_apb_*(apb_*),
        .cmd_*(cmd_*),
        .rsp_*(rsp_*)
    );

    // Address decoder for multiple register banks
    logic [2:0] bank_select;
    logic [2:0] bank_cmd_valid;
    logic [2:0] bank_rsp_valid;

    assign bank_select = cmd_paddr[31:29]; // Top 3 bits for bank
selection

    // Command distribution
    for (genvar i = 0; i < 3; i++) begin : gen_banks
        assign bank_cmd_valid[i] = cmd_valid && (bank_select == i);

        register_bank #(.BANK_ID(i)) u_bank (
            .clk(clk),
            .resetn(resetn),
            .cmd_valid(bank_cmd_valid[i]),
            .cmd_ready(bank_cmd_ready[i]),
            .cmd_*(cmd_*),
            .rsp_valid(bank_rsp_valid[i]),
            .rsp_*(rsp_*)
        );
    end

    // Response arbitration
    assign cmd_ready = |bank_cmd_ready;
    assign rsp_valid = |bank_rsp_valid;

endmodule
```

8.10.2 Error Handling and Status Reporting

```
// Enhanced backend with error handling
module enhanced_register_block (
    input  logic        clk,
    input  logic        resetn,

    // APB slave interface
    input  logic        cmd_valid,
```

```
    output logic        cmd_ready,
    input  logic        cmd_write,
    input  logic [15:0] cmd_addr,
    input  logic [31:0] cmd_wdata,
    input  logic [3:0]  cmd_strb,
    input  logic [2:0]  cmd_prot,

    output logic        rsp_valid,
    input  logic        rsp_ready,
    output logic [31:0] rsp_rdata,
    output logic        rsp_error
);

    // Error conditions
    logic addr_error, prot_error, access_error;

    assign addr_error = (cmd_addr >= 16*4);                    // Out
of range
    assign prot_error = (cmd_prot[0] == 1'b1);                 //
Instruction access
    assign access_error = addr_error || prot_error;

    // Register access with error checking
    always_ff @(posedge clk) begin
        if (cmd_valid && cmd_ready) begin
            if (cmd_write && !access_error) begin
                // Safe register write
                for (int i = 0; i < 4; i++) begin
                    if (cmd_strb[i]) begin
                        registers[cmd_addr[5:2]][i*8+:8] <=
cmd_wdata[i*8+:8];
                    end
                end
            end
        end
    end

    // Response with comprehensive error reporting
    always_ff @(posedge clk) begin
        if (!resetn) begin
            rsp_valid <= 1'b0;
            rsp_rdata <= 32'h0;
            rsp_error <= 1'b0;
        end else if (cmd_valid && cmd_ready) begin
            rsp_valid <= 1'b1;
            rsp_error <= access_error;
```

```
            if (access_error) begin
                rsp_rdata <= 32'hDEADBEEF;  // Error pattern
            end else begin
                rsp_rdata <= cmd_write ? 32'h0 :
registers[cmd_addr[5:2]];
            end
        end else if (rsp_ready) begin
            rsp_valid <= 1'b0;
        end
    end

    assign cmd_ready = !rsp_valid || rsp_ready;

endmodule
```

8.11 Performance Optimization

8.11.1 Buffer Depth Selection

Choose buffer depths based on backend characteristics:

Backend Type	Recommended DEPTH	Rationale
Register Block	2 (4 entries)	Single-cycle response
SRAM Controller	3 (8 entries)	Multi-cycle memory latency
External Memory	4 (16 entries)	High latency, variable timing
DMA Controller	4-5 (16-32 entries)	Burst operations

8.11.2 Clock Domain Optimization

For different clock domains, use `apb_slave_cdc`:

```
apb_slave_cdc #(
    .ADDR_WIDTH(32),
    .DATA_WIDTH(32)
) u_cdc_slave (
    // APB clock domain
    .s_pclk(apb_clk),
    .s_presetn(apb_resetn),
    .s_apb_*(apb_*),

    // Backend clock domain
```

```
    .m_pclk(backend_clk),
    .m_presetn(backend_resetn),
    .cmd_*(backend_cmd_*),
    .rsp_*(backend_rsp_*)
);
```

8.12 Synthesis Considerations

8.12.1 Area Optimization

- Reduce DEPTH for area-constrained designs
- Share skid buffers across multiple slaves when possible
- Use synchronous reset for smaller area

8.12.2 Timing Optimization

- Register all command/response outputs
- Use appropriate buffer depths to meet timing
- Consider pipeline stages for high-frequency operation

8.12.3 Power Optimization

- Use clock-gated variant (`apb_slave_cg`) when available
- Implement conditional clock enables for inactive slaves
- Size buffers appropriately to minimize switching activity

8.13 Verification Notes

8.13.1 Protocol Compliance

- Verify APB setup and access phase timing
- Check PREADY assertion with valid PRDATA/PSLVERR
- Validate proper state machine operation

8.13.2 Buffer Verification

- Test buffer overflow/underflow conditions
- Verify command/response packet integrity
- Check flow control under various load conditions

8.13.3 Backend Integration

- Test various backend response latencies
- Verify error propagation and handling

- Check concurrent transaction handling

8.14 Related Modules

- **apb_slave_cg**: Clock-gated version for power optimization
- **apb_slave_cdc**: Clock domain crossing variant
- **apb_master**: Complementary APB master implementation
- **gaxi_skid_buffer**: Underlying buffering infrastructure
- **apb_xbar**: APB crossbar for multi-slave systems

The `apb_slave` module provides a complete, high-performance solution for APB slave functionality with advanced buffering, full protocol compliance, and flexible backend integration capabilities.

8.15 Navigation

- **← Back to RTLAmba Index**
- **← Back to Main Documentation Index**

9 apb_slave_stub

A lightweight APB slave stub module that provides packed command/response interfaces for simplified testbench integration and system-level testing.

9.1 Overview

The `apb_slave_stub` module acts as a simple APB slave that converts APB protocol signals to packed command/response packets. It's designed for testbench use where a simple APB slave is needed without the full functionality of the standard `apb_slave` module. The packed interface simplifies integration with test responders and verification components.

9.2 Module Declaration

```
module apb_slave_stub #(
    parameter int DEPTH    = 4,
    parameter int DATA_WIDTH    = 32,
    parameter int ADDR_WIDTH    = 32,
    parameter int STRB_WIDTH    = DATA_WIDTH / 8,
    parameter int CMD_PACKET_WIDTH = ADDR_WIDTH + DATA_WIDTH +
STRB_WIDTH + 4, // addr, data, strb, prot, pwrite
    parameter int RESP_PACKET_WIDTH = DATA_WIDTH + 1, // data, resp
    // Short Parameters
```

```
    parameter int DW  = DATA_WIDTH,
    parameter int AW  = ADDR_WIDTH,
    parameter int SW  = STRB_WIDTH,
    parameter int CPW = CMD_PACKET_WIDTH,
    parameter int RPW = RESP_PACKET_WIDTH
) (
    // Clock and Reset
    input  logic                        pclk,
    input  logic                        presetn,

    // APB Slave Interface
    input  logic                        s_apb_PSEL,
    input  logic                        s_apb_PENABLE,
    input  logic [AW-1:0]               s_apb_PADDR,
    input  logic                        s_apb_PWRITE,
    input  logic [DW-1:0]               s_apb_PWDATA,
    input  logic [SW-1:0]               s_apb_PSTRB,
    input  logic [2:0]                  s_apb_PPROT,
    output logic [DW-1:0]               s_apb_PRDATA,
    output logic                        s_apb_PSLVERR,
    output logic                        s_apb_PREADY,

    // Command Packet Interface (packed)
    output logic                        cmd_valid,
    input  logic                        cmd_ready,
    output logic [CPW-1:0]              cmd_data,

    // Response Packet Interface (packed)
    input  logic                        rsp_valid,
    output logic                        rsp_ready,
    input  logic [RPW-1:0]              rsp_data
);
```

9.3 Parameters

Parameter	Type	Default	Description
DEPTH	int	4	Internal buffering depth
DATA_WIDTH	int	32	APB data bus width
ADDR_WIDTH	int	32	APB address bus width
STRB_WIDTH	int	DATA_WIDTH/	Write strobe

Parameter	Type	Default	Description
		8	width (calculated)
CMD_PACKET_WIDTH	int	Calculated	Command packet width
RESP_PACKET_WIDTH	int	Calculated	Response packet width

9.4 Ports

9.4.1 Clock and Reset

Port	Width	Direction	Description
pclk	1	Input	APB clock
presetn	1	Input	APB active-low reset

9.4.2 APB Slave Interface

Port	Width	Direction	Description
s_apb_PSEL	1	Input	Peripheral select
s_apb_PENABLE	1	Input	Enable signal
s_apb_PADDR	AW	Input	Address bus
s_apb_PWRITE	1	Input	Write/read (1=write, 0=read)
s_apb_PWDATA	DW	Input	Write data
s_apb_PSTRB	SW	Input	Write strobe
s_apb_PPROT	3	Input	Protection attributes
s_apb_PRDATA	DW	Output	Read data
s_apb_PSLVERR	1	Output	Slave error
s_apb_PREADY	1	Output	Ready signal

Port	Width	Direction	Description

9.4.3 Packed Command Interface

Port	Width	Direction	Description
cmd_valid	1	Output	Command packet valid
cmd_ready	1	Input	Ready for command packet
cmd_data	CPW	Output	Packed command (pwrite, pprot, pstrb, pwdata, paddr)

9.4.4 Packed Response Interface

Port	Width	Direction	Description
rsp_valid	1	Input	Response packet valid
rsp_ready	1	Output	Ready for response packet
rsp_data	RPW	Input	Packed response (resp, prdata)

9.5 Functional Description

9.5.1 Packet Interface

The stub uses packed interfaces to simplify testbench integration:

Command Packet Format (MSB to LSB): - `pwrite` (1 bit): Write/read operation - `pprot` (3 bits): Protection attributes - `pstrb` (SW bits): Write strobe - `pwdata` (DW bits): Write data - `paddr` (AW bits): Address

Response Packet Format (MSB to LSB): - `pslverr` (1 bit): Slave error - `prdata` (DW bits): Read data

9.5.2 Operation

1. APB master drives transaction on APB interface
2. Stub detects PSEL/PENABLE and packs command

3. Stub presents command packet on `cmd_data` with `cmd_valid=1`
4. Test responder reads command when `cmd_ready=1`
5. Test responder provides response on `rsp_data` with `rsp_valid=1`
6. Stub unpacks response and drives APB PRDATA/PSLVERR/PREADY

9.6 Usage Example

```
// Instantiate APB slave stub
apb_slave_stub #(
    .DATA_WIDTH(32),
    .ADDR_WIDTH(16)
) u_apb_slave_stub (
    .pclk(clk),
    .presetn(rst_n),
    // APB interface from master/interconnect
    .s_apb_PSEL(apb_psel),
    .s_apb_PENABLE(apb_penable),
    .s_apb_PADDR(apb_paddr),
    .s_apb_PWRITE(apb_pwrite),
    .s_apb_PWDATA(apb_pwdata),
    .s_apb_PSTRB(apb_pstrb),
    .s_apb_PPROT(apb_pprot),
    .s_apb_PRDATA(apb_prdata),
    .s_apb_PSLVERR(apb_pslverr),
    .s_apb_PREADY(apb_pready),
    // Packed interface to test responder
    .cmd_valid(test_cmd_valid),
    .cmd_ready(test_cmd_ready),
    .cmd_data(test_cmd_data),
    .rsp_valid(test_rsp_valid),
    .rsp_ready(test_rsp_ready),
    .rsp_data(test_rsp_data)
);

// Example: Respond to command (in testbench)
// When cmd_valid: unpack command, generate response
always_comb begin
    test_cmd_ready = 1'b1;  // Always ready
    if (test_cmd_valid) begin
        // Unpack command
        logic [AW-1:0] addr = test_cmd_data[AW-1:0];
        logic [DW-1:0] wdata = test_cmd_data[AW+DW-1:AW];
        logic pwrite = test_cmd_data[CPW-1];

        // Generate response
        test_rsp_data = {1'b0, read_memory(addr)};  // No error,
```

```
return data
        test_rsp_valid = 1'b1;
    end
end
```

9.7 Design Notes

9.7.1 Testbench Usage

This module is intended for: - System-level testbenches - Integration testing - Simple APB slave emulation - Memory model integration - Verification component integration

For production use, consider the full-featured `apb_slave` module.

9.7.2 Response Generation

The stub expects immediate response from the test driver. For multi-cycle latency: - Test driver should buffer commands - Assert `rsp_valid` only when response is ready - Stub will hold PREADY low until response arrives

9.7.3 Packed Interface Benefits

- Simplified connection to test drivers
- Reduces signal count
- Easier integration with verification IP
- Convenient for memory models and responders

9.8 Related Modules

- `apb_slave.sv` - Full-featured APB slave with independent interfaces
- `apb_master_stub.sv` - Companion APB master stub
- `apb_monitor.sv` - APB transaction monitoring

9.9 References

- **APB Protocol**: AMBA APB Protocol Specification v2.0
- **Full APB Slave**: apb_slave.md
- **APB Master Stub**: apb_master_stub.md

9.10 Navigation

- **← Back to APB Index** (if exists, otherwise remove this line)
- **← Back to RTLAmba Index**

- ← Back to Main Documentation Index

10 apb_xbar

An Advanced Peripheral Bus (APB) crossbar switch that provides full connectivity between multiple APB masters and slaves with weighted round-robin arbitration, programmable address decoding, and high-performance transaction routing.

10.1 Overview

The `apb_xbar` module implements a complete APB crossbar interconnect supporting multiple masters accessing multiple slaves through intelligent arbitration and address-based routing. It features configurable address decoding, weighted round-robin arbitration for fairness control, and comprehensive buffering to optimize system throughput in complex multi-master, multi-slave APB environments.

10.2 Module Declaration

```
module apb_xbar #(
    // Number of APB masters (from the master)
    parameter int M = 3,
    // Number of APB slaves (to the dest)
    parameter int S = 6,
    // Address width
    parameter int ADDR_WIDTH = 32,
    // Data width
    parameter int DATA_WIDTH = 32,
    // Strobe width
    parameter int STRB_WIDTH = DATA_WIDTH/8,
    parameter int MAX_THRESH = 16,
    parameter int DEPTH = 4,
    // Local abbreviations
    parameter int DW     = DATA_WIDTH,
    parameter int AW     = ADDR_WIDTH,
    parameter int SW     = STRB_WIDTH,
    parameter int MID    = $clog2(M),
    parameter int SID    = $clog2(S),
    parameter int MTW    = $clog2(MAX_THRESH),
    parameter int MXMTW  = M * MTW,
    parameter int SXMTW  = S * MTW,
    parameter int SLVCPW = AW + DW + SW + 4,
    parameter int SLVRPW = DW + 1,
    parameter int MSTCPW = AW + DW + SW + 6,
    parameter int MSTRPW = DW + 3
) (
    input  logic                          pclk,
```

```
    input  logic                           presetn,

    // Slave enable for addr decoding
    input  logic [S-1:0]                   SLAVE_ENABLE,
    // Slave address base
    input  logic [S-1:0][ADDR_WIDTH-1:0] SLAVE_ADDR_BASE,
    // Slave address limit
    input  logic [S-1:0][ADDR_WIDTH-1:0] SLAVE_ADDR_LIMIT,
    // Thresholds for the Weighted Round Robin Arbiters
    input  logic [MXMTW-1:0]               MST_THRESHOLDS,
    input  logic [SXMTW-1:0]               SLV_THRESHOLDS,

    // Master interfaces - These are from the APB master
    input  logic [M-1:0]                   m_apb_psel,
    input  logic [M-1:0]                   m_apb_penable,
    input  logic [M-1:0]                   m_apb_pwrite,
    input  logic [M-1:0][2:0]              m_apb_pprot,
    input  logic [M-1:0][ADDR_WIDTH-1:0] m_apb_paddr,
    input  logic [M-1:0][DATA_WIDTH-1:0] m_apb_pwdata,
    input  logic [M-1:0][STRB_WIDTH-1:0] m_apb_pstrb,
    output logic [M-1:0]                   m_apb_pready,
    output logic [M-1:0][DATA_WIDTH-1:0] m_apb_prdata,
    output logic [M-1:0]                   m_apb_pslverr,

    // Slave interfaces - these are to the APB destinations
    output logic [S-1:0]                   s_apb_psel,
    output logic [S-1:0]                   s_apb_penable,
    output logic [S-1:0]                   s_apb_pwrite,
    output logic [S-1:0][2:0]              s_apb_pprot,
    output logic [S-1:0][ADDR_WIDTH-1:0] s_apb_paddr,
    output logic [S-1:0][DATA_WIDTH-1:0] s_apb_pwdata,
    output logic [S-1:0][STRB_WIDTH-1:0] s_apb_pstrb,
    input  logic [S-1:0]                   s_apb_pready,
    input  logic [S-1:0][DATA_WIDTH-1:0] s_apb_prdata,
    input  logic [S-1:0]                   s_apb_pslverr
);
```

10.3 Parameters

Parameter	Type	Default	Description
M	int	3	Number of APB masters (input side)
S	int	6	Number of APB slaves

Parameter	Type	Default	Description
			(output side)
ADDR_WIDTH	int	32	APB address bus width
DATA_WIDTH	int	32	APB data bus width
STRB_WIDTH	int	DATA_WIDTH/8	APB write strobe width
MAX_THRESH	int	16	Maximum arbitration threshold
DEPTH	int	4	Internal buffer depth (2^DEPTH entries)

10.4 Ports

10.4.1 Clock and Reset

Port	Width	Direction	Description
pclk	1	Input	APB clock
presetn	1	Input	APB active-low reset

10.4.2 Configuration Interface

Port	Width	Direction	Description
SLAVE_ENABLE	S	Input	Per-slave enable bits
SLAVE_ADDR_BASE	S × ADDR_WIDTH	Input	Base address for each slave
SLAVE_ADDR_LIMIT	S × ADDR_WIDTH	Input	Limit address for each slave
MST_THRE	M × MTW	Input	Master arbitration

Port	Width	Direction	Description
SHOLDS			thresholds
SLV_THRESHOLDS	S × MTW	Input	Slave arbitration thresholds

10.4.3 Master Interfaces (Input Side)

Port	Width	Direction	Description
m_apb_psel	M	Input	Master select signals
m_apb_penable	M	Input	Master enable signals
m_apb_pwrite	M	Input	Master write/read indicators
m_apb_pprot	M × 3	Input	Master protection attributes
m_apb_paddr	M × ADDR_WIDTH	Input	Master addresses
m_apb_pwdata	M × DATA_WIDTH	Input	Master write data
m_apb_pstrb	M × STRB_WIDTH	Input	Master write strobes
m_apb_pready	M	Output	Master ready signals
m_apb_prdata	M × DATA_WIDTH	Output	Master read data
m_apb_pslverr	M	Output	Master error signals

10.4.4 Slave Interfaces (Output Side)

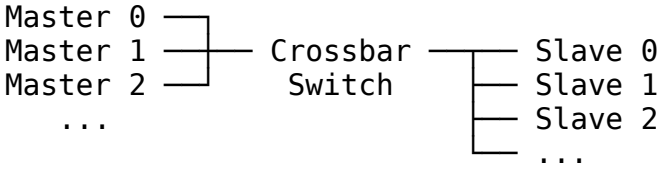
Port	Width	Direction	Description
s_apb_psel	S	Output	Slave select signals
s_apb_penable	S	Output	Slave enable

Port	Width	Direction	Description
			signals
s_apb_pwrite	S	Output	Slave write/read indicators
s_apb_pprot	S × 3	Output	Slave protection attributes
s_apb_paddr	S × ADDR_WIDTH	Output	Slave addresses
s_apb_pwdata	S × DATA_WIDTH	Output	Slave write data
s_apb_pstrb	S × STRB_WIDTH	Output	Slave write strobes
s_apb_pready	S	Input	Slave ready signals
s_apb_prdata	S × DATA_WIDTH	Input	Slave read data
s_apb_pslverr	S	Input	Slave error signals

10.5 Architecture

10.5.1 Crossbar Topology

The crossbar provides full connectivity between all masters and slaves:



10.5.2 Key Components

1. **APB Slave Stubs**: Convert APB protocol to command/response interface (master side)
2. **APB Master Stubs**: Convert command/response interface to APB protocol (slave side)

3. **Address Decoders**: Route transactions based on configurable address ranges
4. **Round-Robin Arbiters**: Uses proven `arbiter_round_robin` module with ACK-based transaction locking
5. **Side Queues**: Buffer master IDs for response routing
6. **Multiplexers/Demultiplexers**: Route commands and responses

Note: Generated crossbars (1to1, 2to1, 1to4, 2to4) use the proven `arbiter_round_robin` module from `rtl/common/` with WAIT_GNT_ACK=1 mode for reliable arbitration.

10.5.3 Data Flow

```
APB Masters → Slave Stubs → Command Queues → Address Decode →
Arbitration →
Master Stubs → APB Slaves

APB Slaves → Master Stubs → Response Queues → ID Matching →
Arbitration →
Slave Stubs → APB Masters
```

10.6 Functionality

10.6.1 Address Decoding

Each slave is assigned an address range defined by: - **SLAVE_ADDR_BASE[i]**: Starting address for slave i - **SLAVE_ADDR_LIMIT[i]**: Ending address for slave i - **SLAVE_ENABLE[i]**: Enable bit for slave i

Address matching logic:

```
if (SLAVE_ENABLE[s] &&
    (addr >= SLAVE_ADDR_BASE[s]) &&
    (addr <= SLAVE_ADDR_LIMIT[s])) begin
    // Route to slave s
end
```

10.6.2 Round-Robin Arbitration with ACK Mode

Generated crossbars use the `arbiter_round_robin` module from `rtl/common/`: - **WAIT_GNT_ACK=1**: Grant locks until transaction completes - **Grant ACK**: `grant && rsp_valid && rsp_ready` signals transaction completion - **Request Vector**: Built from `cmd_valid` and address match (for multi-slave) - **Fair Rotation**: Priority rotates after each transaction

Note: The full `apb_xbar` module with weighted thresholds is separate from the generated crossbars. Generated crossbars (1to1, 2to1, 1to4, 2to4) use standard round-robin for simplicity and reliability.

10.6.3 Transaction Buffering

Internal queues provide: 1. **Command Buffering**: Stores pending transactions 2. **Response Buffering**: Stores completed transactions 3. **ID Tracking**: Maintains master-slave associations

10.7 Timing Characteristics

10.7.1 Latency Components

Component	Latency	Description
Stub Conversion	1 cycle	APB to command/response
Address Decode	0 cycles	Combinatorial logic
Arbitration	1 cycle	Grant generation
Buffer Transit	1 cycle	Queue traversal
Total Minimum	**3 cycles**	Best case path

10.7.2 Throughput Metrics

Metric	Value	Conditions
Maximum Frequency	200-400 MHz	Technology dependent
Peak Throughput	1 transaction/cycle/path	No conflicts
Sustained Throughput	85-95% of peak	With typical arbitration
Concurrent Transactions	Up to M×S	All paths active

10.8 Usage Examples

10.8.1 Basic Multi-Core System

```
apb_xbar #(
    .M(4),                      // 4 CPU cores
    .S(8),                      // 8 peripheral slaves
    .ADDR_WIDTH(32),
    .DATA_WIDTH(32),
    .DEPTH(3)                   // 8-entry buffers
) u_system_xbar (
```

```
    .pclk           (apb_clk),
    .presetn        (apb_resetn),

    // Address map configuration
    .SLAVE_ENABLE    (slave_enable),
    .SLAVE_ADDR_BASE ({
        32'h4000_7000,  // Slave 7: High-speed UART
        32'h4000_6000,  // Slave 6: SPI controller
        32'h4000_5000,  // Slave 5: I2C controller
        32'h4000_4000,  // Slave 4: GPIO controller
        32'h4000_3000,  // Slave 3: Timer 3
        32'h4000_2000,  // Slave 2: Timer 2
        32'h4000_1000,  // Slave 1: Timer 1
        32'h4000_0000   // Slave 0: System controller
    }),
    .SLAVE_ADDR_LIMIT ({
        32'h4000_7FFF,  // Slave 7: 4KB space
        32'h4000_6FFF,  // Slave 6: 4KB space
        32'h4000_5FFF,  // Slave 5: 4KB space
        32'h4000_4FFF,  // Slave 4: 4KB space
        32'h4000_3FFF,  // Slave 3: 4KB space
        32'h4000_2FFF,  // Slave 2: 4KB space
        32'h4000_1FFF,  // Slave 1: 4KB space
        32'h4000_0FFF   // Slave 0: 4KB space
    }),

    // Equal priority arbitration
    .MST_THRESHOLDS  ({4{4'h4}}),  // Equal priority for all masters
    .SLV_THRESHOLDS  ({8{4'h4}}),  // Equal priority for all slaves

    // Master interfaces (from CPUs)
    .m_apb_psel      (cpu_apb_psel),
    .m_apb_penable   (cpu_apb_penable),
    .m_apb_pwrite    (cpu_apb_pwrite),
    .m_apb_pprot     (cpu_apb_pprot),
    .m_apb_paddr     (cpu_apb_paddr),
    .m_apb_pwdata    (cpu_apb_pwdata),
    .m_apb_pstrb     (cpu_apb_pstrb),
    .m_apb_pready    (cpu_apb_pready),
    .m_apb_prdata    (cpu_apb_prdata),
    .m_apb_pslverr   (cpu_apb_pslverr),

    // Slave interfaces (to peripherals)
    .s_apb_psel      (periph_apb_psel),
    .s_apb_penable   (periph_apb_penable),
    .s_apb_pwrite    (periph_apb_pwrite),
    .s_apb_pprot     (periph_apb_pprot),
```

```
    .s_apb_paddr     (periph_apb_paddr),
    .s_apb_pwdata    (periph_apb_pwdata),
    .s_apb_pstrb     (periph_apb_pstrb),
    .s_apb_pready    (periph_apb_pready),
    .s_apb_prdata    (periph_apb_prdata),
    .s_apb_pslverr   (periph_apb_pslverr)
);
```

10.8.2 High-Performance Computing System

```
// HPC system with priority-based arbitration
apb_xbar #(
    .M(6),                  // Multiple masters with different
priorities
    .S(12),                 // Large number of slaves
    .ADDR_WIDTH(32),
    .DATA_WIDTH(64),        // Wide data path
    .DEPTH(4),              // Large buffers for latency tolerance
    .MAX_THRESH(32)         // High threshold range
) u_hpc_xbar (
    .pclk           (hpc_clk),
    .presetn        (hpc_resetn),

    // Hierarchical address map
    .SLAVE_ENABLE    (12'h00F),       // Enable first 12 slaves
    .SLAVE_ADDR_BASE ({
        32'h5000_B000,  // Slave 11: Debug interface
        32'h5000_A000,  // Slave 10: Performance counters
        32'h5000_9000,  // Slave 9:  DMA controller 3
        32'h5000_8000,  // Slave 8:  DMA controller 2
        32'h5000_7000,  // Slave 7:  DMA controller 1
        32'h5000_6000,  // Slave 6:  Memory controller
        32'h5000_5000,  // Slave 5:  Network interface
        32'h5000_4000,  // Slave 4:  Accelerator 3
        32'h5000_3000,  // Slave 3:  Accelerator 2
        32'h5000_2000,  // Slave 2:  Accelerator 1
        32'h5000_1000,  // Slave 1:  Cache controller
        32'h5000_0000   // Slave 0:  System control
    }),
    .SLAVE_ADDR_LIMIT ({
        32'h5000_BFFF,  // 4KB each
        32'h5000_AFFF,
        32'h5000_9FFF,
        32'h5000_8FFF,
        32'h5000_7FFF,
        32'h5000_6FFF,
        32'h5000_5FFF,
        32'h5000_4FFF,
        32'h5000_3FFF,
```

```
        32'h5000_2FFF,
        32'h5000_1FFF,
        32'h5000_0FFF
    }),

    // Priority-based arbitration
    .MST_THRESHOLDS  ({
        4'hF,  // Master 5: Highest priority (system)
        4'hC,  // Master 4: High priority (real-time)
        4'h8,  // Master 3: Medium priority (compute)
        4'h8,  // Master 2: Medium priority (compute)
        4'h4,  // Master 1: Low priority (background)
        4'h2   // Master 0: Lowest priority (debug)
    }),
    .SLV_THRESHOLDS  ({12{4'h8}}),   // Equal slave priority

    // Connect interfaces
    .m_apb_*(master_apb_*),
    .s_apb_*(slave_apb_*)
);
```

10.8.3 Mixed Latency System

```
// System with mix of low-latency and high-throughput requirements
module mixed_latency_system (
    input logic apb_clk,
    input logic apb_resetn,

    // Real-time masters (low latency)
    axi_apb_if.slave  rt_masters [2],
    // Bulk transfer masters (high throughput)
    axi_apb_if.slave  bulk_masters [2],

    // Critical slaves (low latency)
    axi_apb_if.master critical_slaves [4],
    // Memory slaves (high throughput)
    axi_apb_if.master memory_slaves [4]
);

    apb_xbar #(
        .M(4),          // 2 RT + 2 bulk
        .S(8),          // 4 critical + 4 memory
        .DEPTH(2)       // Small buffers for low latency
    ) u_mixed_xbar (
        .pclk(apb_clk),
        .presetn(apb_resetn),
```

```
        // Time-sensitive address ranges
        .SLAVE_ADDR_BASE ({
            32'h6000_7000,  // Memory 3
            32'h6000_6000,  // Memory 2
            32'h6000_5000,  // Memory 1
            32'h6000_4000,  // Memory 0
            32'h6000_3000,  // Critical 3: Interrupt controller
            32'h6000_2000,  // Critical 2: Timer
            32'h6000_1000,  // Critical 1: Real-time IO
            32'h6000_0000   // Critical 0: System control
        }),

        // Latency-optimized arbitration
        .MST_THRESHOLDS ({
            4'h2,   // Master 3: Bulk (low priority)
            4'h2,   // Master 2: Bulk (low priority)
            4'hF,   // Master 1: RT (highest priority)
            4'hF    // Master 0: RT (highest priority)
        }),
        .SLV_THRESHOLDS ({
            4'h4, 4'h4, 4'h4, 4'h4,  // Memory slaves: medium
            4'hF, 4'hF, 4'hF, 4'hF   // Critical slaves: highest
        }),

        // Interface connections
        .m_apb_*(concat({rt_masters, bulk_masters})),
        .s_apb_*(concat({critical_slaves, memory_slaves}))
    );

endmodule
```

10.9 Advanced Configuration

10.9.1 Dynamic Address Mapping

```
// Reconfigurable address decoder
logic [7:0][31:0] dynamic_addr_base;
logic [7:0][31:0] dynamic_addr_limit;
logic [7:0]       dynamic_enable;

always_ff @(posedge apb_clk) begin
    if (config_write && config_addr[15:12] == 4'h1) begin
        case (config_addr[11:0])
            12'h000: dynamic_addr_base[0]  <= config_wdata;
            12'h004: dynamic_addr_limit[0] <= config_wdata;
            12'h008: dynamic_enable[0]     <= config_wdata[0];
            // ... additional configuration registers
```

```
        endcase
    end
end

apb_xbar #(.M(3), .S(8))
u_dynamic_xbar (
    // ...
    .SLAVE_ADDR_BASE (dynamic_addr_base),
    .SLAVE_ADDR_LIMIT(dynamic_addr_limit),
    .SLAVE_ENABLE    (dynamic_enable),
    // ...
);
```

10.9.2 Performance Monitoring

```
// Transaction performance monitoring
logic [31:0] transaction_counts [M][S];
logic [31:0] arbitration_cycles [M];
logic [31:0] total_transactions;

always_ff @(posedge apb_clk) begin
    for (int m = 0; m < M; m++) begin
        for (int s = 0; s < S; s++) begin
            if (transaction_complete[m][s]) begin
                transaction_counts[m][s] <= transaction_counts[m][s] +
1;
                total_transactions <= total_transactions + 1;
            end
        end

        if (arbitration_active[m] && !arbitration_grant[m]) begin
            arbitration_cycles[m] <= arbitration_cycles[m] + 1;
        end
    end
end

// Performance metrics
assign avg_arbitration_delay = arbitration_cycles /
total_transactions;
assign master_utilization = transaction_counts / total_cycles;
```

10.9.3 Quality of Service (QoS)

```
// QoS-aware threshold management
logic [3:0] qos_thresholds [M];
logic [3:0] dynamic_mst_thresh [M];

always_comb begin
```

```
    for (int m = 0; m < M; m++) begin
        case (master_qos[m])
            2'b11: dynamic_mst_thresh[m] = 4'hF;  // Critical
            2'b10: dynamic_mst_thresh[m] = 4'hC;  // High
            2'b01: dynamic_mst_thresh[m] = 4'h8;  // Medium
            2'b00: dynamic_mst_thresh[m] = 4'h4;  // Low
        endcase
    end
end

apb_xbar u_qos_xbar (
    // ...
    .MST_THRESHOLDS({dynamic_mst_thresh[3], dynamic_mst_thresh[2],
                     dynamic_mst_thresh[1], dynamic_mst_thresh[0]}),
    // ...
);
```

10.10 Performance Optimization

10.10.1 Buffer Sizing Guidelines

System Type	Recommended DEPTH	Buffer Size	Benefit
Low Latency	2	4 entries	Minimal delay
Balanced	3-4	8-16 entries	Good throughput/latency
High Throughput	4-5	16-32 entries	Maximum bandwidth
Variable Latency	5-6	32-64 entries	Latency tolerance

10.10.2 Arbitration Tuning

```
// Example threshold configurations
localparam [3:0] CRITICAL_THRESH = 4'hF;  // 15/16 cycles
localparam [3:0] HIGH_THRESH     = 4'hC;  // 12/16 cycles
localparam [3:0] NORMAL_THRESH   = 4'h8;  // 8/16 cycles
localparam [3:0] LOW_THRESH      = 4'h4;  // 4/16 cycles
localparam [3:0] BACKGROUND_THRESH = 4'h2;  // 2/16 cycles
```

10.10.3 Address Map Optimization

```
// Optimize for cache line alignment
localparam [31:0] SLAVE_BASES [8] = '{
```

```
    32'h4000_0000,  // Align to 64KB boundaries
    32'h4001_0000,  // for optimal cache behavior
    32'h4002_0000,
    32'h4003_0000,
    32'h4004_0000,
    32'h4005_0000,
    32'h4006_0000,
    32'h4007_0000
};
```

10.11 Synthesis Considerations

10.11.1 Area Optimization

- Reduce M and S parameters to minimum required
- Use smaller DEPTH values for area-constrained designs
- Share arbiters when timing allows

10.11.2 Timing Optimization

- Register all crossbar outputs
- Use pipeline stages for critical paths
- Balance buffer depths with frequency requirements

10.11.3 Power Optimization

- Use clock gating on unused master/slave ports
- Implement threshold-based power scaling
- Gate clocks during idle periods

10.12 Verification Notes

10.12.1 Connectivity Verification

- Verify all M×S path combinations
- Check address decoding for all ranges
- Validate arbitration fairness

10.12.2 Protocol Compliance

- Check APB protocol adherence on all ports
- Verify proper PREADY handling
- Validate error propagation

10.12.3 Performance Verification

- Measure throughput under various loads
- Check arbitration latency and fairness
- Verify buffer efficiency

10.13 Related Modules

- **apb_xbar_thin**: Lightweight crossbar for simple configurations
- **apb_master**: APB master for crossbar output side (cmd/rsp to APB)
- **apb_slave**: APB slave for crossbar input side (APB to cmd/rsp)
- **arbiter_round_robin**: Proven round-robin arbiter used in generated crossbars
- **arbiter_priority_encoder**: Dependency of arbiter_round_robin
- **encoder**: Used for address decoding in multi-slave crossbars
- **apb_monitor**: APB protocol monitoring and debugging

Generated Crossbar Dependencies: - `rtl/amba/gaxi/gaxi_fifo_sync.sv` (used by apb_slave/apb_master) - `rtl/amba/gaxi/gaxi_skid_buffer.sv` (used by apb_slave/apb_master) - `rtl/common/arbiter_round_robin.sv` (arbitration) - `rtl/common/arbiter_priority_encoder.sv` (arbiter dependency) - `rtl/common/encoder.sv` (multi-slave address decode)

The `apb_xbar` module provides a complete, high-performance solution for multi-master, multi-slave APB systems with advanced arbitration, flexible address decoding, and comprehensive system integration capabilities.

10.14 Navigation

- **← Back to RTLAmba Index**
- **← Back to Main Documentation Index**

11 APB (Advanced Peripheral Bus) Modules

Location: `rtl/amba/apb/` **Test Location:** `val/amba/` **Status:** √ Production Ready

---

11.1 Overview

The APB subsystem provides a complete implementation of the ARM AMBA APB (Advanced Peripheral Bus) protocol, including masters, slaves, monitors, interconnect components, and testbench utilities.

APB is a simple, low-power peripheral bus designed for connecting low-bandwidth peripherals to a system bus. It uses a simple two-cycle handshake protocol with minimal control signals.

11.2 Module Categories

11.2.1 Core Protocol Components

Module	Description	Documentation	Status
apb_master	Full-featured APB master with command/response interface	apb_master.md	✓ Documented
apb_slave	Complete APB slave with buffered cmd/rsp interface	apb_slave.md	✓ Documented
apb_slave_cdc	APB slave with clock domain crossing support	apb_slave_cdc.md	✓ Documented
apb_monitor	Transaction monitoring with 128-bit monitor bus + 64-bit timestamp	apb_monitor.md	✓ Documented

11.2.2 Testbench Utilities

Module	Description	Documentation	Status
apb_master_stub	Lightweight APB master for testbench integration	apb_master_stub.md	✓ Documented
apb_slave_stub	Lightweight APB slave for testbench integration	apb_slave_stub.md	✓ Documented

11.2.3 Interconnect Components

Module	Description	Documentation	Status
apb_crossbar	Full APB crossbar interconnect (NxM topology)	apb_crossbar.md	✓ Documented
apb_xbar	APB crossbar with address decoding	apb_xbar.md	✓ Documented

11.2.4 Coverage Variants

Module	Description	Documentation	Status
apb_master_cg	APB master with integrated coverage groups	-	⏳ Pending
apb_slave_cg	APB slave with integrated coverage groups	-	⏳ Pending
apb_slave_cdc_cg	APB slave CDC with integrated coverage groups	-	⏳ Pending

11.3 Key Features

11.3.1 APB Protocol Support

- ✓ **Full APB4/APB5 Compliance:** Complete protocol implementation
- ✓ **PSTRB Support:** Byte-lane strobes for partial writes
- ✓ **PPROT Support:** Protection attributes for security-aware systems
- ✓ **Error Handling:** PSLVERR support for error responses

11.3.2 Clock Domain Crossing

- ✓ **Dual-Clock Operation:** APB (pclk) and backend (aclk) domains
- ✓ **Safe CDC:** Proper handshake-based clock domain crossing
- ✓ **Independent Frequencies:** Backend can run faster or slower than APB

11.3.3 Monitoring and Debug

- ✓ **Transaction Monitoring:** Real-time protocol monitoring

- ✓ **64-bit Monitor Bus:** Standardized packet format
- ✓ **Error Detection:** Protocol violations, timeout detection
- ✓ **Performance Tracking:** Transaction counting, latency measurement

11.3.4 Testbench Integration

- ✓ **Packed Interfaces:** Simplified testbench connectivity
- ✓ **Stub Modules:** Lightweight wrappers for CocoTB integration
- ✓ **WaveDrom Support:** Automated waveform generation

11.4 Quick Start

11.4.1 Using APB Master

```
apb_master #(
    .ADDR_WIDTH(32),
    .DATA_WIDTH(32),
    .DEPTH(2)
) u_apb_master (
    .aclk           (clk),
    .aresetn        (resetn),

    // Command interface
    .cmd_valid      (cmd_valid),
    .cmd_ready      (cmd_ready),
    .cmd_pwrite     (cmd_pwrite),
    .cmd_paddr      (cmd_paddr),
    .cmd_pwdata     (cmd_pwdata),
    .cmd_pstrb      (cmd_pstrb),

    // Response interface
    .rsp_valid      (rsp_valid),
    .rsp_ready      (rsp_ready),
    .rsp_prdata     (rsp_prdata),
    .rsp_pslverr    (rsp_pslverr),

    // APB master interface
    .m_apb_PSEL     (psel),
    .m_apb_PENABLE  (penable),
    .m_apb_PREADY   (pready),
    .m_apb_PADDR    (paddr),
    .m_apb_PWRITE   (pwrite),
    .m_apb_PWDATA   (pwdata),
    .m_apb_PSTRB    (pstrb),
```

```
    .m_apb_PRDATA   (prdata),
    .m_apb_PSLVERR  (pslverr)
);
```

11.4.2 Using APB Slave

```
apb_slave #(
    .ADDR_WIDTH(32),
    .DATA_WIDTH(32),
    .DEPTH(2)
) u_apb_slave (
    .aclk           (clk),
    .aresetn        (resetn),

    // APB slave interface
    .s_apb_PSEL     (psel),
    .s_apb_PENABLE  (penable),
    .s_apb_PREADY   (pready),
    .s_apb_PADDR    (paddr),
    .s_apb_PWRITE   (pwrite),
    .s_apb_PWDATA   (pwdata),
    .s_apb_PSTRB    (pstrb),
    .s_apb_PRDATA   (prdata),
    .s_apb_PSLVERR  (pslverr),

    // Command interface
    .cmd_valid      (cmd_valid),
    .cmd_ready      (cmd_ready),
    .cmd_pwrite     (cmd_pwrite),
    .cmd_paddr      (cmd_paddr),
    .cmd_pwdata     (cmd_pwdata),
    .cmd_pstrb      (cmd_pstrb),

    // Response interface
    .rsp_valid      (rsp_valid),
    .rsp_ready      (rsp_ready),
    .rsp_prdata     (rsp_prdata),
    .rsp_pslverr    (rsp_pslverr)
);
```

11.5 Testing

All APB modules are verified using CocoTB-based testbenches located in `val/amba/`:

```
# Run all APB tests
pytest val/amba/test_apb*.py -v
```

```
# Run specific module tests
pytest val/amba/test_apb_master.py -v
pytest val/amba/test_apb_slave.py -v
pytest val/amba/test_apb_slave_cdc.py -v
pytest val/amba/test_apb_monitor.py -v

# Run with waveform generation
env ENABLE_WAVEDROM=1 pytest val/amba/test_apb_slave_wavedrom.py -v
```

11.5.1 WaveDrom Tests

Several modules have dedicated WaveDrom tests that generate detailed timing diagrams:

- `test_apb_slave_wavedrom.py` - APB slave protocol waveforms
- `test_apb_slave_cdc.py::test_apb_slave_cdc_wavedrom` - CDC timing diagrams

Generated waveforms are stored in: - JSON format: `_wavedrom/` - SVG images: `_wavedrom_svg/`

11.6 Protocol Details

11.6.1 APB Transfer Phases

APB uses a simple two-phase protocol:

1. **SETUP Phase:** PSEL asserted, PENABLE low
   - Master presents address, control signals, and write data (if write)
   - Slave prepares for transfer
2. **ACCESS Phase:** PSEL and PENABLE both asserted
   - Slave completes transfer and asserts PREADY when ready
   - For reads, slave presents PRDATA
   - Slave may assert PSLVERR to signal error

11.6.2 Signal Descriptions

Signal	Direction	Description
PCLK	Input	APB clock
PRESETn	Input	Active-low reset
PSEL	Master→Slave	Slave select
PENABLE	Master→Slave	Enable (ACCESS

Signal	Direction	Description
		phase indicator)
PREADY	Slave→Master	Transfer complete
PADDR	Master→Slave	Address bus
PWRITE	Master→Slave	Write enable (1=write, 0=read)
PWDATA	Master→Slave	Write data
PSTRB	Master→Slave	Byte lane strobes
PPROT	Master→Slave	Protection attributes
PRDATA	Slave→Master	Read data
PSLVERR	Slave→Master	Error response

11.7 Design Notes

11.7.1 Command/Response Architecture

All APB modules use a command/response interface pattern for backend integration:

Command Interface (Master → Backend or APB → Backend): - `cmd_valid`, `cmd_ready` - Handshake signals - `cmd_pwrite` - Write/read direction - `cmd_paddr` - Transaction address - `cmd_pwdata` - Write data - `cmd_pstrb` - Byte strobes - `cmd_pprot` - Protection attributes

Response Interface (Backend → Master or Backend → APB): - `rsp_valid`, `rsp_ready` - Handshake signals - `rsp_prdata` - Read data - `rsp_pslverr` - Error flag

This separation enables: - Clean clock domain crossing - Buffering and pipelining - Easy testbench integration - Backend processing flexibility

11.7.2 Monitor Bus Protocol

The APB monitor outputs standardized 128-bit `monitor_packet_t` records, paired with a 64-bit side-band timestamp:

```
[127:124] - Packet Type    (4 bits)
[123:109] - Reserved       (15 bits, forward-compat slack)
[108:105] - Protocol       (4 bits)
[104:97]  - Event Code     (8 bits)
[96:88]   - Channel ID     (9 bits)
[87:72]   - Agent ID       (16 bits)
[71:64]   - Unit ID        (8 bits)
[63:0]    - Event Data     (64 bits)
```

See apb_monitor.md for detailed packet format.

11.8 Related Documentation

- **AXI4 Modules** - Full AXI4 protocol components
- **AXIL4 Modules** - AXI4-Lite components
- **AXIS4 Modules** - AXI4-Stream components
- **GAXI Modules** - Generic AXI utilities

11.9 References

11.9.1 Specifications

- ARM AMBA APB Protocol Specification v2.0
- ARM AMBA 4 APB Protocol Specification

11.9.2 Source Code

- RTL: `rtl/amba/apb/`
- Tests: `val/amba/test_apb*.py`
- Framework: `bin/TBClasses/components/apb/`

11.9.3 Test Documentation

- APB Master Tests: `val/amba/test_apb_master.py`
- APB Slave Tests: `val/amba/test_apb_slave.py`
- APB CDC Tests: `val/amba/test_apb_slave_cdc.py`
- APB Monitor Tests: `val/amba/test_apb_monitor.py`

Last Updated: 2025-10-20

11.10 Navigation

- **← Back to RTLAmba Index**
- **← Back to Main Documentation Index**